

VICTORIA PARK ACADEMY SHENZHEN

CIE IGCSE COMPUTER SCIENCE (0478) - YEAR 9 SEMESTER 2

WEEKS 13-18: FILE HANDLING, TESTING, PROGRAM DEVELOPMENT & MOCK EXAM

Course Context:

Students have completed Weeks 1-12 covering: Algorithms, Pseudocode, Flowcharts, Searching (linear/binary), Python fundamentals (variables, I/O, operators, selection, iteration, 1D/2D arrays, strings, functions, scope).

This block (Weeks 13-18) covers:

Week 13: Reading from Files

Week 14: Writing to Files

Week 15: Testing - Dry Runs & Trace Tables

Week 16: Program Development Lifecycle

Week 17: Revision Clinic

Week 18: Mock Exam & Review

Class Profile: EAL learners, mixed ability, 40-minute lessons

IDE: Thonny (Python 3.x)

WEEK 13: READING FROM FILES

LESSON 13.1 — INTRODUCTION TO FILE HANDLING: READING FILES

Topic: Reading data from text files in Python

Duration: 40 minutes

Lesson Type: Direct Instruction + Practical

Syllabus Ref: CIE 0478 Section 2.2 (Programming), 2.2.3 File Handling

Learning Objective:

By the end of this lesson, students will be able to open and read text files in Python using `open()`, `read()`, `readline()`, and `readlines()`, and handle `FileNotFoundError` using `try/except` blocks.

Key Vocabulary:

- file handling (文件处理 / wen jian chu li)
- open a file (打开文件 / da kai wen jian)
- read mode (读取模式 / du qu mo shi) — represented as 'r'
- text file (文本文件 / wen ben wen jian)

- file path (文件路径 / wen jian lu jing)
- line by line (逐行 / zhu hang)
- error handling (错误处理 / cuo wu chu li)
- exception (异常 / yi chang)
- try/except (尝试/捕获 / chang shi / bu huo)
- FileNotFoundError (文件未找到错误 / wen jian wei zhao dao cuo wu)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

Display on board: "Write down 3 reasons why a program might need to read data from a file instead of asking the user to type it."

Students write answers in notebooks. Teacher circulates and reads out good answers.

3-12 min | DIRECT INSTRUCTION

Teacher explains:

- Why file handling? Data persistence (data survives after program ends), processing large datasets, sharing data between programs.
- The open() function: fileObject = open("filename.txt", "r")
- Modes: "r" = read, "w" = write, "a" = append
- The file path: absolute vs relative paths. For Thonny, place the text file in the same folder as the Python program.
- Four ways to read:

1. read() — reads entire file as one string
2. readline() — reads one line at a time
3. readlines() — reads all lines into a list
4. for line in file — loops through file line by line (best for large files)

- ALWAYS close the file: fileObject.close()
- Better: use "with" statement (auto-closes):

WITH open("file.txt", "r") AS myFile:

```
data = myFile.read()
```

- Error handling: what if the file doesn't exist?

TRY

```
OPEN "data.txt" FOR READ
```

```
EXCEPT FileNotFoundError
```

```
OUTPUT "File not found!"
```

END TRY

Live code demo on projector (Thonny):

```
# Method 1: read() - entire file
myFile = open("students.txt", "r")
content = myFile.read()
print(content)
myFile.close()

# Method 2: with statement (recommended)
with open("students.txt", "r") as myFile:
    content = myFile.read()
    print(content)

# Method 3: read line by line
with open("students.txt", "r") as myFile:
    for line in myFile:
        print(line.strip()) # strip removes \n
```

Create "students.txt" on projector with 5 names.

12-25 min | GUIDED PRACTICE

Students open Thonny. Teacher provides "students.txt" file with:

Alice Chen

Bob Wang

Carol Li

David Zhang

Eva Wu

Task 1 (all): Write a program to open the file and print all names.

Task 2 (all): Write a program to count how many students are in the file (HINT: use a counter variable inside the loop).

Teacher circulates, checks code, helps with file path issues.

Common issue: file not in same folder — show students how to check.

25-35 min | INDEPENDENT PRACTICE

Task: Write a program that reads "students.txt" and:

- Prints each name with a number (1. Alice Chen, 2. Bob Wang...)
- At the end, prints "Total students: X"

Extension: Add error handling with try/except so if the file is missing, the program prints "Error: Could not find students.txt" instead of crashing.

35-40 min | PLENARY + HOMEWORK

Plenary: Cold-call 2 students to show their output. Discuss:

"Why is the for line in file method better than read() for large

files?" (Answer: read() loads everything into RAM; line by line is memory efficient.)

Homework: Complete Worksheet 13A (trace table + code reading).

EAL Support:

- Word bank on board with all key terms in English, Chinese, pinyin
- Sentence starters for discussions: "I think file handling is important because..."
- Visual diagram showing file flow: Text File -> open() -> Program -> Output
- Pair EAL students with stronger English speakers during guided practice
- Provide printed glossary at start of lesson

Differentiation:

Emerging: Use scaffolded code with blanks to fill in. Provide pre-written students.txt. Focus only on the with statement method. Teacher check-in at 5-minute intervals.

Developing: Complete all tasks independently. Add try/except error handling. Support partner at their table if needed.

Secure: Complete all tasks + extension. Research and implement the readlines() method as an alternative. Help debug peers' code. Explain the difference between read(), readline(), and readlines() to a partner.

Resources Needed:

- Projector + Thonny
- students.txt file (shared via USB/cloud)
- Laptops with Thonny installed
- Printed vocabulary list
- Worksheet 13A (homework)

--- LESSON 13.2 ---

LESSON 13.2 — PROCESSING DATA FROM FILES

Topic: Splitting lines, converting data types, processing CSV files

Duration: 40 minutes

Lesson Type: Direct Instruction + Practical

Syllabus Ref: CIE 0478 Section 2.2.3 (File Handling), 2.2.2 (Data Types)

Learning Objective:

By the end of this lesson, students will be able to read a CSV file, split lines by comma, convert string data to numeric types, and calculate statistics (such as average) from file data.

Key Vocabulary:

- CSV file (逗号分隔值文件 / dou hao fen ge zhi wen jian)
- comma separated values (逗号分隔值 / dou hao fen ge zhi)
- split (分割 / fen ge)
- data processing (数据处理 / shu ju chu li)
- convert type (类型转换 / lei xing zhuan huan)
- integer (整数 / zheng shu)
- float (浮点数 / fu dian shu)
- string method (字符串方法 / zi fu chuan fang fa)
- sum (求和 / qiu he)
- average / mean (平均值 / ping jun zhi)
- counter (计数器 / ji shu qi)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

Display: "A file contains: 'Alice,85,90,78'. Write one line of

Python code to split this string into a list."

(Answer: data = "Alice,85,90,78".split(","))

Quick check — thumbs up when done.

3-12 min | DIRECT INSTRUCTION

Teacher explains:

- CSV files: a simple way to store data in tables using commas.

Common format for spreadsheets and databases.

- Example marks.csv:

Name,Math,Science,English

Alice,85,90,78

Bob,72,88,91

Carol,95,82,85

- Reading and splitting:

with open("marks.csv", "r") as file:

for line in file:

data = line.strip().split(",")

name = data[0]

math = int(data[1]) # Convert string to integer!

- Converting types: int() for whole numbers, float() for decimals.

If you forget to convert, you'll get string concatenation errors!

- Calculating average:

total = math + science + english

```
average = total / 3
```

- Skipping the header row: use a boolean flag or check line number.

Live code demo:

Read marks.csv and calculate each student's average

with open("marks.csv", "r") as file:

```
    header = file.readline() # Skip header
```

```
    for line in file:
```

```
        data = line.strip().split(",")
```

```
        name = data[0]
```

```
        math = int(data[1])
```

```
        science = int(data[2])
```

```
        english = int(data[3])
```

```
        average = (math + science + english) / 3
```

```
        print(name + ": " + str(average))
```

12-25 min | GUIDED PRACTICE

Students receive marks.csv with 8 students and 3 subjects.

Task 1: Write a program to read the file and print each student's name with their average mark.

Task 2: Calculate and print the CLASS average for each subject.

(HINT: You'll need running totals for each subject and a counter.)

Starter code provided for Emerging students:

with open("marks.csv", "r") as file:

```
    header = file.readline() # Skip header
```

```
    math_total = 0
```

```
    count = 0
```

```
    for line in file:
```

```
        data = line.strip().split(",")
```

```
        name = _____
```

```
        math = int(data[1])
```

```
        average = _____
```

```
        print(name + ": " + str(average))
```

```
        math_total = math_total + math
```

```
        count = count + 1
```

```
    class_math_avg = _____
```

```
    print("Class Math Average: " + str(class_math_avg))
```

25-35 min | INDEPENDENT PRACTICE

Task: Write a program that reads marks.csv and:

- Finds and prints the HIGHEST scoring student in Math
- Finds and prints the LOWEST scoring student in English
- Counts how many students scored above 80 in Science

Extension: Use a 2D array (list of lists) to store all data, then print a formatted report.

35-40 min | PLENARY + HOMEWORK

Plenary: Show correct solution on board. Key question: "Why must we use `int()` when reading numbers from a file?" (Answer: file data is always read as strings; must convert for math operations.)

Homework: Worksheet 13B — trace table + code completion.

EAL Support:

- Display CSV file format diagram on board with colour-coded columns
- Bilingual labels on diagram: Name (姓名), Math (数学), Science (科学)
- Provide word bank with data type conversion functions (`int()`, `float()`, `str()`)
- Sentence frame: "To calculate the average, first I _____, then I _____."
- Visual: Show before/after of `.split(",")` operation

Differentiation:

Emerging: Complete Task 1 with heavily scaffolded code (fill-in-the-blanks).

Use only 3 students in CSV. Teacher provides printed pseudocode.

Focus on understanding the `split()` and `int()` steps.

Developing: Complete Tasks 1 and 2 independently. Attempt independent practice.

Add comments to explain each line of code.

Secure: Complete all tasks including extension. Implement full formatted report with subject headers and aligned columns. Research the `csv` module (`import csv`) as an alternative approach.

Resources Needed:

- marks.csv file (8 students x 3 subjects)
- Projector + Thonny
- Printed scaffolded code (Emerging)
- Worksheet 13B (homework)

--- WORKSHEET 13A ---

WORKSHEET 13A — READING FROM FILES

Name: _____ Class: _____ Date: _____

Section A: Vocabulary Matching (8 marks)

Match each term to its correct definition.

- 5. file handling ____ a) A list of folders leading to a file
- 6. read mode ('r') ____ b) Catches and handles program errors
- 7. file path ____ c) Working with data stored in files
- 8. exception ____ d) Opens a file for reading data
- 9. try/except ____ e) An error that occurs during program execution
- 10. read() ____ f) Reads one line from a file
- 11. readline() ____ g) Closes a file automatically
- 12. with statement ____ h) Reads the entire file as one string

Section B: Code Reading (10 marks)

Read the following Python program and answer the questions.

Program to read student names from a file

try:

```
with open("classlist.txt", "r") as myFile:
```

```
    count = 0
```

```
    for line in myFile:
```

```
        name = line.strip()
```

```
        count = count + 1
```

```
        print(str(count) + ". " + name)
```

```
    print("Total students: " + str(count))
```

except FileNotFoundError:

```
    print("Error: The file was not found.")
```

The file "classlist.txt" contains:

Maria Santos

James Park

Lina Ahmed

Kai Tanaka

9. What is the purpose of line.strip()? (2 marks)

13. What will be the EXACT output of this program? Write it below. (4 marks)

14. What would happen if "classlist.txt" did not exist in the folder? (2 marks)

15. Why is the with statement better than using open() and close() separately?
(2 marks)

Section C: Trace Table (8 marks)

16. Complete the trace table for the following program. The file "numbers.txt" contains: 5, 8, 3, 10, 2 (each number on a new line).

with open("numbers.txt", "r") as file:

total = 0

count = 0

for line in file:

num = int(line.strip())

total = total + num

count = count + 1

average = total / count

print("Sum: " + str(total))

print("Average: " + str(average))

Iteration	line (string)	num	total	count	Output
Initial	-	-	0	0	-
1	"5\n"				-
2					-
3					-
4					-
5					-
After loop	-	-			

Section D: Code Completion (9 marks)

17. Fill in the missing code to read a file of countries and print those starting with the letter 'A'.

try:

with open(_____, _____) as file:

for _____ in file:

country = _____.strip()

if _____:

print(_____)

except _____:

print("File not found!")

TOTAL: 35 marks

WORKSHEET 13B — PROCESSING DATA FROM FILES

Name: _____ Class: _____ Date: _____

Section A: Multiple Choice (6 marks)

Circle the correct answer.

1. What does `.split(",")` do to a string?

- a) Removes all commas
- b) Splits the string into a list using commas as separators
- c) Joins strings with commas
- d) Counts the number of commas

18. A file line contains: "Alice,85". How do you get Alice's score as a number?

- a) `int("Alice,85")`
- b) `int("Alice,85".split(",")[0])`
- c) `int("Alice,85".split(",")[1])`
- d) `int("Alice,85".split(","))`

3. Why is `.strip()` used when reading lines from a file?

- a) To delete the file
- b) To remove whitespace including the newline character
- c) To encrypt the data
- d) To convert to uppercase

4. What is the correct way to skip a CSV header row?

- a) `header = file.readline()`
- b) `skip(file.header)`
- c) `file.skip(1)`
- d) `ignore_header = True`

19. Which data type is `data[0]` after `line.strip().split(",")`?

- a) Integer
- b) Float
- c) String
- d) Boolean

20. Given: `values = [10, 20, 30]`. What is `sum(values) / len(values)`?

- a) 60
- b) 3

c) 20.0

d) 10

Section B: Code Analysis (10 marks)

The file "scores.csv" contains:

Name,Score

Amy,78

Ben,92

Cat,65

Dan,88

Eve,71

21. Trace through the following program. Write the EXACT output. (6 marks)

with open("scores.csv", "r") as file:

```
    file.readline() # Skip header
```

```
    highest = 0
```

```
    best_student = ""
```

```
    for line in file:
```

```
        data = line.strip().split(",")
```

```
        name = data[0]
```

```
        score = int(data[1])
```

```
        if score > highest:
```

```
            highest = score
```

```
            best_student = name
```

```
    print("Best student: " + best_student)
```

```
    print("Highest score: " + str(highest))
```

Output:

22. If we wanted to find the LOWEST score instead, which two lines would change and what would they change to? (4 marks)

Section C: Programming Challenge (14 marks)

9. Write a complete Python program that:

a) Opens "scores.csv" (skip the header) (2 marks)

b) Reads each student's name and score (2 marks)

- c) Calculates the class average score (4 marks)
- d) Counts how many students scored 80 or above (4 marks)
- e) Prints both results with clear labels (2 marks)

Write your answer in the space below:

TOTAL: 30 marks

--- ANSWER KEY (Week 13) ---

WORKSHEET 13A — ANSWER KEY

Section A: Vocabulary Matching (8 marks)

- 23.c (file handling → Working with data stored in files)
- 24.d (read mode ('r') → Opens a file for reading data)
- 25.a (file path → A list of folders leading to a file)
- 26.e (exception → An error that occurs during program execution)
- 27.b (try/except → Catches and handles program errors)
- 28.h (read() → Reads the entire file as one string)
- 29.f (readline() → Reads one line from a file)
- 30.g (with statement → Closes a file automatically)

Section B: Code Reading (10 marks)

- 31. line.strip() removes whitespace characters from the beginning and end of the string, including the newline character (\n) that exists at the end of each line in a file. (2 marks — 1 for whitespace, 1 for newline)

10. Exact output: (4 marks — 1 mark per correct line)

- 32. Maria Santos
- 33. James Park
- 34. Lina Ahmed
- 35. Kai Tanaka

Total students: 4

- 36. If the file did not exist, the except FileNotFoundError block would run, and the program would print: "Error: The file was not found." (2 marks)
- 37. The with statement is better because it automatically closes the file when the block ends, even if an error occurs. This prevents memory leaks and file corruption. (2 marks — 1 for auto-close, 1 for error handling)

Section C: Trace Table (8 marks)

Iteration	line (string)	num	total	count	Output
Initial	-	-	0	0	-
1	"5\n"	5	5	1	-
2	"8\n"	8	13	2	-
3	"3\n"	3	16	3	-
4	"10\n"	10	26	4	-

5	"2\n"	2	28	5	-
After loop	-	-	28	5	Sum: 28
After loop	-	-	28	5	Average: 5.6

(1 mark per correct row in iterations 1-5; 2 marks for correct outputs)

Section D: Code Completion (9 marks)

try:

 with open("countries.txt", "r") as file:

 for line in file:

 country = line.strip()

 if country[0] == "A":

 print(country)

except FileNotFoundError:

 print("File not found!")

(1 mark each blank, accept equivalent answers)

WORKSHEET 13B — ANSWER KEY

Section A: Multiple Choice (6 marks)

38. b

39. c

40. b

41. a

42. c

43. c

Section B: Code Analysis (10 marks)

7. Output: (6 marks)

Best student: Ben

Highest score: 92

(3 marks per correct line)

8. To find lowest: (4 marks)

Line "highest = 0" becomes "lowest = 100" (or any high number)

Line "if score > highest:" becomes "if score < lowest:"

(2 marks per correct change, accept equivalent logic)

Section C: Programming Challenge (14 marks)

9. Model answer:

with open("scores.csv", "r") as file:

```
    file.readline() # Skip header (2 marks)
```

```
    total = 0
```

```
    count = 0
```

```
    above_80 = 0
```

```
    for line in file:          # (2 marks for loop structure)
```

```
        data = line.strip().split(",")
```

```
        name = data[0]
```

```
        score = int(data[1])
```

```
        total = total + score
```

```
        count = count + 1
```

```
        if score >= 80:        # (4 marks for counting logic)
```

```
            above_80 = above_80 + 1
```

```
    average = total / count    # (4 marks for average calc)
```

```
    print("Class average: " + str(average))
```

```
    print("Students with 80+: " + str(above_80)) # (2 marks for output)
```

(Alternative approaches accepted if logically correct)

WEEK 14: WRITING TO FILES

LESSON 14.1 — WRITING AND APPENDING TO FILES

Topic: Creating and modifying text files in Python

Duration: 40 minutes

Lesson Type: Direct Instruction + Practical

Syllabus Ref: CIE 0478 Section 2.2.3 (File Handling)

Learning Objective:

By the end of this lesson, students will be able to write data to text files using 'w' (write) and 'a' (append) modes, and explain the difference between overwriting and appending to a file.

Key Vocabulary:

- write mode (写入模式 / xie ru mo shi) — represented as 'w'
- append mode (追加模式 / zhui jia mo shi) — represented as 'a'
- overwrite (覆盖 / fu gai)

- write() method (写入方法 / xie ru fang fa)
- writelines() method (写入多行方法 / xie ru duo xing fang fa)
- newline character (换行符 / huan xing fu) — "\n"
- file pointer (文件指针 / wen jian zhi zhen)
- log file (日志文件 / ri zhi wen jian)
- data persistence (数据持久化 / shu ju chi jiu hua)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

Display: "You want to save your program's output so you can read it tomorrow. Would you use 'r', 'w', or 'a' mode? Explain."

Students discuss with partner for 1 minute, then teacher takes answers. Introduce that today's lesson is about creating files.

3-12 min | DIRECT INSTRUCTION

Teacher explains:

- Reading vs Writing: So far we've only READ from files. Now we CREATE and WRITE to files.
 - Write mode 'w': Opens file for writing. If file exists, it is OVERWRITTEN (all previous data is lost!). If file doesn't exist, a new file is created.
 - Append mode 'a': Opens file for writing. If file exists, new data is ADDED at the end. If file doesn't exist, a new file is created.
- Use 'a' for log files, diaries, and adding data without losing existing content.
- The write() method: file.write("text") — writes a string. Must add "\n" for new lines! Unlike print(), write() does NOT automatically add a newline.
 - The writelines() method: file.writelines(list) — writes all strings in a list. Also needs "\n" in each string.

Live code demo (Thonny on projector):

Method 1: write() — creates or overwrites diary.txt

with open("diary.txt", "w") as file:

```
file.write("My Digital Diary\n")
file.write("=====\n\n")
file.write("Today I learned about file handling.\n")
file.write("It is very useful for saving data.\n")
```

```
print("Diary created!")
```

```
# Method 2: append mode — adds without deleting
with open("diary.txt", "a") as file:
    file.write("\n--- New Entry ---\n")
    file.write("Date: 2025-06-02\n")
    file.write("Mood: Happy\n")
print("Entry added!")

# Method 3: writelines()
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("lines.txt", "w") as file:
    file.writelines(lines)

CRITICAL: Show students what happens if they forget "\n":
with open("bad.txt", "w") as file:
    file.write("First")
    file.write("Second")

# Result in file: FirstSecond (all on one line!)
```

12-25 min | GUIDED PRACTICE

Task 1 (all): Write a program that asks the user for their name and favourite hobby, then writes this information to "profile.txt" in a formatted way.

Task 2 (all): Write a program that asks the user for a diary entry, then APPENDS it to "mydiary.txt" with a timestamp (the date). Run it 3 times to prove the data is being added, not overwritten.

Starter code for Emerging:

```
name = input("Enter your name: ")
hobby = input("Enter your hobby: ")
with open(_____, _____) as file:
    file.write(_____ + "\n")
    file.write(_____ + "\n")
print("Profile saved!")
```

25-35 min | INDEPENDENT PRACTICE

Task: Create a "Simple Logger" program that:

- Displays a menu: 1) Add log entry 2) View all logs 3) Exit
- Option 1: Ask user for a message, append it to "log.txt" with the current time (use import datetime)
- Option 2: Open and display all contents of "log.txt"
- Option 3: Exit the program

Extension: Add error handling — if "log.txt" doesn't exist when trying to view, print "No logs found yet."

35-40 min | PLENARY + HOMEWORK

Plenary: Thumbs up/down/sideways: "Do you understand the difference between 'w' and 'a' mode?" Cold-call a thumbs-up student to explain.

Key question: "When would you use 'w' vs 'a'?"

Homework: Worksheet 14A — writing mode questions and code completion.

EAL Support:

- Visual comparison chart on board: 'r' vs 'w' vs 'a' with icons
- Colour-code the modes: 'r' = green (safe), 'w' = red (danger/overwrite), 'a' = blue (add)
- Sentence starter: "I would use ____ mode when I want to ____ because..."
- Pair discussion protocol: Partner A explains 'w', Partner B explains 'a'
- Provide printed mode reference card

Differentiation:

Emerging: Complete Task 1 with fill-in-the-blanks code. Focus ONLY on 'w' mode. Teacher provides printed menu code structure for guidance.
Verbal explanation of difference between 'w' and 'a' accepted.

Developing: Complete Tasks 1 and 2 independently. Begin independent practice.
Use comments to explain mode choice.

Secure: Complete all tasks + full logger program with menu. Implement error handling for missing files. Research and use datetime module for real timestamps. Explain to a peer when to use each mode.

Resources Needed:

- Projector + Thonny
- Pre-created example files to demonstrate overwrite
- Printed mode reference cards
- Worksheet 14A

--- LESSON 14.2 ---

LESSON 14.2 — CSV FILES AND A SIMPLE DATABASE

Topic: Reading and writing CSV files; building a simple student management system

Duration: 40 minutes

Lesson Type: Project-Based Lesson

Syllabus Ref: CIE 0478 Section 2.2.3 (File Handling), 2.2.4 (Data Structures)

Learning Objective:

By the end of this lesson, students will be able to read and write CSV files,

and create a simple menu-driven program that acts as a database (add records, view records, save to file).

Key Vocabulary:

- database (数据库 / shu ju ku)
- record (记录 / ji lu)
- field (字段 / zi duan)
- menu-driven program (菜单驱动程序 / cai dan qu dong cheng xu)
- delimiter (分隔符 / fen ge fu)
- save to file (保存到文件 / bao cun dao wen jian)
- view records (查看记录 / cha kan ji lu)
- add record (添加记录 / tian jia ji lu)
- comma (逗号 / dou hao)
- user interface (用户界面 / yong hu jie mian)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

Display: "A teacher wants to store student names and grades in a file. Each student has: Name, Age, Math Grade, English Grade."

Write one line showing how you would store this data using commas."

Example answer: "Alice Chen,14,85,92"

Quick check — 2 students read answers aloud.

3-12 min | DIRECT INSTRUCTION

Teacher explains:

- CSV as a simple database: Each line is a RECORD, each comma-separated value is a FIELD.
- Reading CSV: same as before — read, strip, split by comma.
- Writing CSV: join data with commas, add "\n" at the end.
- Formatting output: use string concatenation or f-strings.

Demo — Student Management System structure:

```
# MENU
```

```
print("STUDENT MANAGEMENT SYSTEM")
```

```
print("1. Add Student")
```

```
print("2. View All Students")
```

```
print("3. Exit")
```

```
choice = input("Enter choice: ")
```

```
# ADD STUDENT
```

```
name = input("Enter name: ")
```

```

age = input("Enter age: ")
grade = input("Enter grade: ")
with open("students.csv", "a") as file:
    file.write(name + "," + age + "," + grade + "\n")
print("Student added!")
# VIEW ALL
with open("students.csv", "r") as file:
    for line in file:
        data = line.strip().split(",")
        print("Name: " + data[0] + ", Age: " + data[1] +
              ", Grade: " + data[2])

```

Key design pattern: while loop for menu:

```

choice = ""
WHILE choice != "3"
    PRINT menu
    choice = INPUT
    IF choice == "1" THEN
        CALL add_student()
    ELSE IF choice == "2" THEN
        CALL view_students()
    END IF
END WHILE

```

12-25 min | GUIDED PRACTICE

Students build the student management system step by step.

Step 1: Write a function `add_student()` that asks for name, age, and grade, then appends to "students.csv".

Step 2: Write a function `view_students()` that reads and displays all records in a formatted way.

Step 3: Add a while-loop menu that calls the functions.

Scaffold for Emerging (provided on handout):

```

def add_student():
    name = input("Name: ")
    age = input("Age: ")
    grade = input("Grade: ")
    with open(_____, _____) as file:
        file.write(_____ + "," + _____ + "," +

```

```

        _____ + _____)
    print("Student added!")
def view_students():
    with open(_____, _____) as file:
        for _____ in file:
            data = _____.strip().split(_____)
            print("Name: " + _____)
choice = ""
while choice != _____:
    print("1. Add Student")
    print("2. View Students")
    print("3. Exit")
    choice = input(_____)
    if choice == _____:
        elif choice == _____:
print("Goodbye!")

```

25-35 min | INDEPENDENT PRACTICE

Task: Add these features to your student management system:

- Add a SEARCH function: ask for a name, then display that student's

record (or "Student not found")

- Add a COUNT function: display how many students are in the database
- Add formatted headers when viewing (use string formatting)

Extension: Add a DELETE function (hint: read all data, filter out the student to delete, write all back using 'w' mode).

35-40 min | PLENARY + HOMEWORK

Plenary: Vote — which feature was hardest to implement? Discuss solutions as a class. Emphasise that this is a real-world skill: databases are used everywhere (school records, banking, hospitals).

Homework: Worksheet 14B — CSV writing tasks.

EAL Support:

- Display pseudocode flowchart for menu system on board
- Bilingual menu template: Add (添加), View (查看), Exit (退出)
- Sentence frame: "To add a student, I need to ____ the data and then ____ it to the file."
- Word bank with all file operation verbs in English/Chinese

- Pair programming: one student reads, one types

Differentiation:

Emerging: Complete guided practice with scaffolded code (fill-in-blanks).

Focus on understanding the pattern: read -> process -> write.

Menu uses only 2 options (add and view). Teacher assists with file path issues.

Developing: Complete all 3 guided steps independently. Attempt 2 features from independent practice. Add comments to all code.

Secure: Complete full system with all features + extension (delete).

Implement input validation (check age is a number, grade is valid).

Research the csv module as an alternative to manual split/join.

Resources Needed:

- Projector + Thonny

- Scaffolded code handouts (Emerging)

- students.csv template file

- Worksheet 14B

--- WORKSHEET 14A ---

WORKSHEET 14A — WRITING TO FILES

Name: _____ Class: _____ Date: _____

Section A: Mode Selection (8 marks)

For each scenario, write which file mode ('r', 'w', or 'a') you would use and explain why.

1. Creating a new report file from scratch.

Mode: _____ Explanation: _____

44. Adding a new entry to an existing diary file without losing old entries.

Mode: _____ Explanation: _____

3. Reading student data to calculate averages.

Mode: _____ Explanation: _____

45. Saving program results — you want a fresh file each time the program runs.

Mode: _____ Explanation: _____

46. Updating a log file with timestamps each time an event occurs.

Mode: _____ Explanation: _____

Section B: Code Reading (10 marks)

Read the following program and answer the questions.

with open("notes.txt", "w") as file:

file.write("Shopping List\n")

file.write("-----\n")

items = ["Apples", "Bread", "Milk", "Eggs"]

for item in items:

file.write("- " + item + "\n")

file.write("\nEnd of list")

47. What will be the exact contents of "notes.txt" after this program runs?

Write the complete file contents: (4 marks)

48. If the program is run a second time, what will be in "notes.txt"? (2 marks)

49. How would you change the code to ADD to the existing list instead of replacing it? (2 marks)

50. What would happen if you removed all the "\n" from this code? (2 marks)

Section C: Code Completion (12 marks)

51. Complete the program to create a guestbook that stores visitor names.

def sign_guestbook():

name = input("Enter your name: ")

message = input("Leave a message: ")

with open(_____, _____) as file:

file.write(_____ + ": " + _____ + _____)

print("Thank you for signing!")

def view_guestbook():

try:

with open(_____, _____) as file:

for _____ in file:

print(_____)

except _____:

print("No entries yet.")

Main program

while _____:

print("\n=== GUESTBOOK ===")

print("1. Sign guestbook")

```

print("2. View guestbook")
print("3. Exit")
choice = input("Choice: ")
if choice == "1":
elif choice == "2":
elif choice == "3":
    print("Goodbye!")
else:
    print("Invalid choice.")

```

TOTAL: 30 marks

--- WORKSHEET 14B ---

WORKSHEET 14B — CSV FILES AND DATABASES

Name: _____ Class: _____ Date: _____

Section A: CSV Knowledge Check (8 marks)

1. What does CSV stand for? (1 mark)

52. In the line "Alice,15,92,88", how many fields are there and what does each field represent? (2 marks)

53. Why must you add "\n" when writing to a CSV file but not when using print()? (2 marks)

54. Write ONE line of Python code to join these variables into a CSV format:

name = "Bob", age = 16, score = 85 (1 mark)

55. After line = "Alice,15,92,88".strip().split(","), what is line[2]? (1 mark)

56. What happens if you open an existing CSV file with 'w' mode? (1 mark)

Section B: Trace Table (10 marks)

57. Complete the trace table for the following program. The file starts empty.

with open("data.csv", "w") as file:

```
students = [["Amy", "14", "85"], ["Ben", "15", "92"], ["Cat", "14", "78"]]
```

```
for i in range(3):
```

```
    line = students[i][0] + "," + students[i][1] + "," + students[i][2] + "\n"
```

```
    file.write(line)
```

```
    print("Wrote: " + line.strip())
```

Iteration	i	students[i]	line	Output
Initial	-	-	-	-

1				
2				
3				

After the program runs, what are the exact contents of data.csv? (4 marks)

Section C: Programming Challenge (12 marks)

58. Write a complete Python program for a "Book Library" system that:

- a) Has a menu with 3 options: Add Book, View Books, Exit (2 marks)
- b) "Add Book" asks for title and author, saves to "library.csv" (4 marks)
- c) "View Books" reads and displays all books nicely formatted (4 marks)
- d) "Exit" ends the program (2 marks)

Write your complete program below:

TOTAL: 30 marks

--- ANSWER KEY (Week 14) ---

WORKSHEET 14A — ANSWER KEY

Section A: Mode Selection (8 marks)

- 59. 'w' — Creates a new file or overwrites existing. Need fresh file. (2 marks)
- 60. 'a' — Append mode adds new data without deleting existing entries. (2 marks)
- 61. 'r' — Only need to read data, not modify. (2 marks)
- 62. 'w' — Overwrite ensures no old data remains. (1 mark)
- 63. 'a' — Each new log entry should be added to existing logs. (1 mark)

Section B: Code Reading (10 marks)

6. Exact contents of notes.txt: (4 marks)

Shopping List

- Apples
- Bread
- Milk
- Eggs

End of list

(1 mark deduction per error)

64. It will be exactly the same. 'w' mode overwrites, so the second run produces identical output. (2 marks)

8. Change "w" to "a" on line 1. (2 marks)

65. All text would be on one continuous line without any line breaks.

The file would contain: "Shopping List-----ApplesBreadMilkEggsEnd of list"

(2 marks — 1 for no line breaks, 1 for continuous text)

Section C: Code Completion (12 marks)

```
def sign_guestbook():
    name = input("Enter your name: ")
    message = input("Leave a message: ")
    with open("guestbook.txt", "a") as file:    # (1 mark)
        file.write(name + ": " + message + "\n") # (2 marks)
    print("Thank you for signing!")

def view_guestbook():
    try:
        with open("guestbook.txt", "r") as file: # (1 mark)
            for line in file:                    # (1 mark)
                print(line.strip())              # (1 mark)
    except FileNotFoundError:                    # (1 mark)
        print("No entries yet.")
```

Main program

```
while True:                                # (1 mark)
    print("\n=== GUESTBOOK ===")
    print("1. Sign guestbook")
    print("2. View guestbook")
    print("3. Exit")
    choice = input("Choice: ")
    if choice == "1":
        sign_guestbook()                    # (1 mark)
    elif choice == "2":
        view_guestbook()                    # (1 mark)
    elif choice == "3":
        print("Goodbye!")
        break                                # (1 mark)
    else:
        print("Invalid choice.")
---
```

WORKSHEET 14B — ANSWER KEY

Section A: CSV Knowledge Check (8 marks)

1. Comma Separated Values (1 mark)

66. 4 fields: Name, Age, Score1, Score2 (accept equivalent) (2 marks)

67. `file.write()` does not automatically add newline like `print()` does.

Without `"\n"`, all data would be on one line. (2 marks)

68. `line = name + "," + str(age) + "," + str(score) + "\n"`

(1 mark, accept equivalent)

69. `"92"` (the third field, index 2) (1 mark)

70. All existing data is deleted/overwritten. The file becomes empty before writing. (1 mark)

Section B: Trace Table (10 marks)

Iteration	i	students[i]	line	Output
Initial	-	-	-	-
1	0	["Amy", "14", "85"]	"Amy,14,85\n"	Wrote: Amy,14,85
2	1	["Ben", "15", "92"]	"Ben,15,92\n"	Wrote: Ben,15,92
3	2	["Cat", "14", "78"]	"Cat,14,78\n"	Wrote: Cat,14,78

(1 mark per correct row)

Contents of data.csv: (4 marks)

Amy,14,85

Ben,15,92

Cat,14,78

Section C: Programming Challenge (12 marks)

Model answer:

```
def add_book():
    # (2 marks for structure)
    title = input("Enter book title: ")
    author = input("Enter author: ")
    with open("library.csv", "a") as file:
        # (1 mark for 'a' mode)
        file.write(title + "," + author + "\n")
        # (1 mark)
    print("Book added!")

def view_books():
    try:
        with open("library.csv", "r") as file:
            # (1 mark)
            print("\n--- LIBRARY ---")
            for line in file:
                # (1 mark)
                data = line.strip().split(",")
                # (1 mark)
                print("Title: " + data[0] + ", Author: " + data[1])
                # (1 mark)
```

```
except FileNotFoundError:
    print("No books in library yet.")
```

Main program

(2 marks

for menu loop)

```
choice = ""
while choice != "3":
    print("\n=== BOOK LIBRARY ===")
    print("1. Add Book")
    print("2. View Books")
    print("3. Exit")
    choice = input("Choice: ")
    if choice == "1":
        add_book()
    elif choice == "2":
        view_books()
    elif choice == "3":
        print("Goodbye!")
    else:
        print("Invalid choice.")
---
```

(Alternative valid approaches accepted)

WEEK 15: TESTING — DRY RUNS & TRACE TABLES

LESSON 15.1 — WHY TEST? TYPES OF ERRORS AND TRACE TABLES

Topic: Understanding testing, error types, and completing trace tables

Duration: 40 minutes

Lesson Type: Direct Instruction + Practice

Syllabus Ref: CIE 0478 Section 1.1 (Algorithm Design), 2.3 (Testing)

Learning Objective:

By the end of this lesson, students will be able to classify program errors into syntax, logic, and runtime types, and complete a trace table for a given algorithm with iteration and selection.

Key Vocabulary:

- testing (测试 / ce shi)

- syntax error (语法错误 / yu fa cuo wu)
- logic error (逻辑错误 / luo ji cuo wu)
- runtime error (运行时错误 / yun xing shi cuo wu)
- dry run (手动运行 / shou dong yun xing) — executing code manually
- trace table (追踪表 / zhui zong biao)
- variable column (变量列 / bian liang lie)
- output column (输出列 / shu chu lie)
- iteration (迭代 / die dai)
- condition (条件 / tiao jian)
- bug (程序错误 / cheng xu cuo wu)
- debug (调试 / tiao shi)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

Display three code snippets on board:

A: `print("Hello)` [Missing closing quote]

B: `x = 5 + "3"` [Type error at runtime]

C: `for i in range(5): print(i * 2)` expecting 1,2,3,4,5

Students classify each as syntax, logic, or runtime error.

(Answers: A=syntax, B=runtime, C=logic)

3-12 min | DIRECT INSTRUCTION

Teacher explains:

- Why test? Programs can have errors. Testing finds errors BEFORE

users do. Untested software can be dangerous (medical systems, banking, aircraft).

- Three types of errors:

71. SYNTAX ERROR: Grammar mistake in code. Program won't run at all.

Python shows red underline and error message.

Examples: missing colon, wrong indentation, misspelled keywords.

72. RUNTIME ERROR: Code is grammatically correct but crashes

during execution. Program starts but stops unexpectedly.

Examples: divide by zero, `FileNotFoundError`, `int("hello")`, accessing index out of range.

73. LOGIC ERROR: Code runs without crashing but produces wrong

results. HARDEST to find because program appears to work.

Examples: using `<` instead of `<=`, wrong formula, off-by-one

errors in loops.

- What is a dry run? Manually executing code step by step without

a computer. You track each variable's value through the program.

- What is a trace table? A structured table used during a dry run.

Has columns for each variable and an output column.

Live demo — dry run on projector:

Algorithm:

$x \leftarrow 5$

$y \leftarrow 3$

$total \leftarrow 0$

WHILE $x > y$ DO

$total \leftarrow total + x$

$x \leftarrow x - 1$

END WHILE

OUTPUT total

Trace table built step by step on board:

Iteration	x	y	total	Condition ($x > y$)	Output
Initial	5	3	0	-	-
1	5	3	5	True	-
After	4	3	5	True	-
2	4	3	9	True	-
After	3	3	9	False	-
End	3	3	9	-	9

12-25 min | GUIDED PRACTICE

Task 1 (all): Complete trace table for this algorithm:

$num \leftarrow 1$

$sum \leftarrow 0$

WHILE $num \leq 4$ DO

$sum \leftarrow sum + num$

$num \leftarrow num + 1$

END WHILE

OUTPUT sum

Students work in pairs. Teacher circulates and checks.

Task 2 (all): Complete trace table with selection:

$a \leftarrow 10$

```

b ← 7
IF a > b THEN
    result ← a - b
ELSE
    result ← b - a
END IF
OUTPUT result

```

25-35 min | INDEPENDENT PRACTICE

Task: Complete the trace table for this algorithm independently:

```

n ← 6
factorial ← 1
WHILE n > 0 DO
    factorial ← factorial * n
    n ← n - 1
END WHILE
OUTPUT factorial

```

Extension: Write the Python code that matches this algorithm, then run it in Thonny to verify your trace table is correct.

35-40 min | PLENARY + HOMEWORK

Plenary: Quick-fire round — teacher shows an error description, students call out the type.

"Missing bracket" → SYNTAX

"Program gives wrong answer but runs" → LOGIC

"File not found while program is running" → RUNTIME

Homework: Worksheet 15A — Error types and trace tables.

EAL Support:

- Colour-code error types: Syntax = red (stop sign), Runtime = yellow (warning), Logic = orange (confusing)
- Visual icons for each error type on board
- Trace table template provided on all handouts
- Sentence frames: "This is a ___ error because ___"
- Bilingual error type descriptions on wall poster

Differentiation:

Emerging: Focus only on identifying error types (not trace tables in detail). Complete Task 1 with teacher-led step-by-step on board. Use pre-printed trace table with only some cells to fill.

Verbal explanation of what a trace table shows accepted.

Developing: Complete all tasks independently. Begin extension task.

Explain to a partner how to complete a trace table.

Secure: Complete all tasks + extension. Create your own algorithm with a loop and selection, then build its trace table. Swap with a partner and check each other's work. Identify which error types are most common in your own coding.

Resources Needed:

- Projector + board
- Pre-printed trace table templates
- Error type posters (Syntax/Runtime/Logic)
- Worksheet 15A

--- LESSON 15.2 ---

LESSON 15.2 — TEST PLANS AND TEST DATA

Topic: Designing test plans with normal, boundary, and erroneous data

Duration: 40 minutes

Lesson Type: Direct Instruction + Practice

Syllabus Ref: CIE 0478 Section 2.3 (Testing)

Learning Objective:

By the end of this lesson, students will be able to design a test plan with normal, boundary (extreme), and erroneous (invalid) test data, and explain why each type is necessary.

Key Vocabulary:

- test plan (测试计划 / ce shi ji hua)
- test data (测试数据 / ce shi shu ju)
- normal data (正常数据 / zheng chang shu ju) — valid, typical input
- boundary data (边界数据 / bian jie shu ju) — extreme valid values
- erroneous data (错误数据 / cuo wu shu ju) — invalid input
- extreme data (极端数据 / ji duan shu ju)
- invalid data (无效数据 / wu xiao shu ju)
- expected output (预期输出 / yu qi shu chu)
- actual output (实际输出 / shi ji shu chu)
- test case (测试用例 / ce shi yong li)
- validation (验证 / yan zheng)
- verification (确认 / que ren)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

"You are testing a program that asks for your exam score (0-100).

Write down 3 different scores you would test with and explain why."

Students write individually. Teacher selects 2-3 to share.

Introduce: today we learn the formal names for these test types.

3-12 min | DIRECT INSTRUCTION

Teacher explains:

- Why test plans matter: A test plan is a document that lists all the tests you will run on a program. It ensures you test THOROUGHLY — not just once with easy data.
- Three types of test data:

74. NORMAL data: Valid, typical input. What the user normally enters. Example: Score = 75 (a typical exam score).

Purpose: Check the program works for standard cases.

75. BOUNDARY/EXTREME data: Valid values at the limits.

Example: Score = 0, Score = 100 (minimum and maximum).

Purpose: Check edge cases where errors often occur.

76. ERRONEOUS/INVALID data: Invalid input that should be rejected.

Example: Score = -5, Score = 150, Score = "hello".

Purpose: Check the program handles bad input gracefully.

- Test plan format (Cambridge standard):

Test #	Description	Input Data	Expected Output	Type	
1	Normal score	75	"Grade: B"	Normal	
2	Minimum boundary	0	"Grade: F"	Boundary	
3	Maximum boundary	100	"Grade: A*"	Boundary	
4	Negative number	-5	"Invalid input"	Erroneous	
5	Above maximum	150	"Invalid input"	Erroneous	

- The boundary between grades is where errors often hide:
 - 69 vs 70 (C vs B boundary)
 - 79 vs 80 (B vs A boundary)
 - Exactly on the boundary must be tested!

12-25 min | GUIDED PRACTICE

Scenario: A program asks the user to enter their age (1-120) and prints a ticket price:

- Age 0-4: Free
- Age 5-17: Child fare (\$5)

- Age 18-64: Adult fare (\$10)
- Age 65+: Senior fare (\$7)

Task (all): In pairs, complete a test plan with at least:

- 2 normal tests
- 3 boundary tests (at each boundary: 4/5, 17/18, 64/65, and 0, 120)
- 2 erroneous tests

Teacher provides template and circulates to check.

25-35 min | INDEPENDENT PRACTICE

Task: Write a complete test plan for this specification:

"A program asks for a student's mark (0-100) and awards a grade:

90-100 = A*, 80-89 = A, 70-79 = B, 60-69 = C, 50-59 = D,

below 50 = F. The program should reject marks outside 0-100."

Must include:

- At least 2 normal tests
- At least 4 boundary tests (at grade boundaries)
- At least 2 erroneous tests
- All tests must have expected outputs

Extension: Write pseudocode for this grade calculator, then use your test plan to dry-run it and verify expected outputs.

35-40 min | PLENARY + HOMEWORK

Plenary: Display a student's test plan on board (with permission).

Class critiques: "Are all boundary values tested? Are erroneous

cases covered?" Key question: "Why test boundaries specifically?"

(Answer: programmers often use wrong operators like < instead of <=, causing off-by-one errors at boundaries.)

Homework: Worksheet 15B — Test plan design.

EAL Support:

- Test plan template provided in both English and Chinese headers
- Visual boundary diagram: number line showing grade ranges with highlighted boundary points
- Colour-coded test types: Normal = green, Boundary = yellow, Erroneous = red
- Sentence frame: "I chose ____ as my ____ test because it tests ____"
- Worked example provided as reference throughout lesson

Differentiation:

Emerging: Complete guided practice with teacher support. Focus on normal and boundary tests only (skip erroneous). Use pre-filled template

with descriptions already written — just fill in input data and expected output. Teacher provides 1:1 check at end.

Developing: Complete guided practice independently. Complete all test types in independent practice. Add reasoning column to test plan.

Secure: Complete all tasks + extension. Research and add "robustness testing" to the test plan. Create a test plan for a file-handling program (testing for empty files, missing files, wrong formats). Explain to the class why boundary testing is critical.

Resources Needed:

- Projector + board
- Test plan template handouts
- Number line diagram for boundaries
- Grade boundary reference chart
- Worksheet 15B

--- WORKSHEET 15A ---

WORKSHEET 15A — ERRORS AND TRACE TABLES

Name: _____ Class: _____ Date: _____

Section A: Error Classification (9 marks)

For each scenario, write the error type (syntax, logic, or runtime) and explain your answer.

1. A Python program has: `for i in range(5) print(i)`

Error type: _____

Explanation: _____

77. A program calculates $\text{area} = \text{length} + \text{width}$ instead of $\text{length} * \text{width}$.

The program runs but gives wrong answers.

Error type: _____

Explanation: _____

78. A program tries to open "data.txt" but the file doesn't exist.

Python shows: `FileNotFoundError`.

Error type: _____

Explanation: _____

79. A program has: `while x < 10:` with no change to `x` inside the loop.

The program runs forever.

Error type: _____

Explanation: _____

5. A student writes: `if x = 5:` instead of `if x == 5:`

Error type: _____

Explanation: _____

80. A program divides total by count, but count is 0 because no data was entered.

Error type: _____

Explanation: _____

81. A loop runs 4 times instead of 5: `for i in range(4)` should be `range(5)`.

Error type: _____

Explanation: _____

8. `IndentationError: unexpected indent` on line 12.

Error type: _____

Explanation: _____

82. A sorting algorithm puts items in reverse order by mistake.

Error type: _____

Explanation: _____

Section B: Trace Table — Loop (10 marks)

10. Complete the trace table for the following algorithm:

`num ← 2`

`result ← 1`

`WHILE num <= 5 DO`

`result ← result * num`

`num ← num + 1`

`END WHILE`

`OUTPUT result`

Iteration	num	result	num <= 5?	Output
Initial			-	-
1				
End			-	

Section C: Trace Table — Selection + Loop (10 marks)

11. Complete the trace table for the following algorithm:

```

values ← [4, 7, 2, 9, 3]
count ← 0
FOR i FROM 0 TO 4 DO
    IF values[i] > 5 THEN
        count ← count + 1
        OUTPUT values[i] + " is big"
    ELSE
        OUTPUT values[i] + " is small"
    END IF
END FOR
OUTPUT "Total big numbers: " + count
---
```

Iteration	i	values[i]	values[i]>5?	count	Output
Initial	-	-	-		-
1					
2					
3					
4					
5					
After	-	-	-		

Section D: Python Trace Table (6 marks)

12. Trace the following Python code:

```

---
total = 0
for i in range(1, 6):
    if i % 2 == 0:
        total = total + i
    else:
        total = total - i
print(total)
---
```

Iteration	i	i % 2 == 0?	total	Output
Initial	-	-		-
1				
2				
3				

4				
5				
End	-	-		

TOTAL: 35 marks

--- WORKSHEET 15B ---

WORKSHEET 15B — TEST PLANS

Name: _____ Class: _____ Date: _____

Section A: Test Data Types (6 marks)

1. Match each test data type to its description:

Normal data ___ a) Input at the minimum or maximum valid value

Boundary data ___ b) Invalid input that should be rejected

Erroneous data ___ c) Valid, typical input that the user normally enters

83. Give ONE example of each test data type for a program that asks for a person's height in cm (valid range: 50-250).

Normal: _____

Boundary: _____

Erroneous: _____

3. Why is boundary data particularly important? (2 marks)

Section B: Test Plan Completion (14 marks)

84. A program asks for a password and checks if it is at least 8 characters long. Complete the test plan:

Test #	Description	Input Data	Expected Output	Type
1	Valid password	"computer123"	"Password accepted"	Normal
2	Minimum length			
3	Empty password			
4	7 characters			
5	Very long password	"a" * 100		Normal

85. A program asks for a day number (1-7) and prints the day name (1=Monday, 2=Tuesday, ..., 7=Sunday). Complete this test plan:

Test #	Description	Input	Expected Output	Type
1	First day	1	"Monday"	
2	Middle day	4		
3	Last day			
4	Too low			

5	Too high	8		
6	Decimal	3.5		
7	Zero			

Section C: Design Your Own Test Plan (15 marks)

6. Design a complete test plan for this specification:

"A program reads a student's mark (0-100). If the mark is 90-100, print 'A*'. If 80-89, print 'A'. If 70-79, print 'B'. If 60-69, print 'C'. If 50-59, print 'D'. Below 50, print 'F'. If the mark is outside 0-100, print 'Invalid mark'."

Your test plan must include:

- At least 2 normal tests
- At least 4 boundary tests (at each grade boundary)
- At least 3 erroneous tests
- All tests must have input data and expected output

Test #	Description	Input	Expected Output	Type
--------	-------------	-------	-----------------	------

TOTAL: 35 marks

--- ANSWER KEY (Week 15) ---

WORKSHEET 15A — ANSWER KEY

Section A: Error Classification (9 marks)

86. Syntax error — Missing colon after for loop. Python expects a colon before the indented block. (1 mark)

87. Logic error — The program runs but produces wrong results. The formula is incorrect (should be multiplication, not addition). (1 mark)

88. Runtime error — FileNotFoundError occurs during program execution. The code is syntactically correct but the file doesn't exist. (1 mark)

89. Logic error — Infinite loop is a logic error. The condition never becomes false because x never changes. (1 mark)

90. Syntax error — Using = (assignment) instead of == (comparison) in an if statement causes a syntax error in Python. (1 mark)

91. Runtime error — Division by zero occurs at runtime. ZeroDivisionError. The code is syntactically correct. (1 mark)

92. Logic error — Off-by-one error. Program runs but doesn't iterate the correct number of times, giving wrong results. (1 mark)

93. Syntax error — IndentationError is a type of syntax error in Python. Wrong indentation prevents the program from running. (1 mark)

94. Logic error — The program runs but produces incorrectly sorted output.

The algorithm logic is wrong. (1 mark)

Section B: Trace Table — Loop (10 marks)

Iteration	num	result	num <= 5?	Output
Initial	2	1	-	-
1	2	2	True	-
After	3	2	True	-
2	3	6	True	-
After	4	6	True	-
3	4	24	True	-
After	5	24	True	-
4	5	120	True	-
After	6	120	False	-
End	6	120	-	120

(1 mark per correct row, 2 marks for final output)

Section C: Trace Table — Selection + Loop (10 marks)

Iteration	i	values[i]	values[i]>5?	count	Output
Initial	-	-	-	0	-
1	0	4	False	0	4 is small
2	1	7	True	1	7 is big
3	2	2	False	1	2 is small
4	3	9	True	2	9 is big
5	4	3	False	2	3 is small
After	-	-	-	2	Total big numbers: 2

(1 mark per correct row, 2 marks for final output)

Section D: Python Trace Table (6 marks)

Iteration	i	i % 2 == 0?	total	Output
Initial	-	-	0	-
1	1	False	-1	-
2	2	True	1	-
3	3	False	-2	-
4	4	True	2	-
5	5	False	-3	-
End	-	-	-3	-3

(1 mark per row, final output = -3)

WORKSHEET 15B — ANSWER KEY

Section A: Test Data Types (6 marks)

1. Normal data → c (Valid, typical input)

Boundary data → a (Input at minimum or maximum valid value)

Erroneous data → b (Invalid input that should be rejected)

(1 mark each)

95. Examples for height (50-250): (2 marks — 1 for each valid example)

Normal: 170 (or any value 50-250)

Boundary: 50 or 250

Erroneous: 30 (or any value < 50, or > 250, or non-numeric)

3. Boundary data is important because: (2 marks)

- Programmers often make off-by-one errors (using < instead of <=)
- Boundary values are where conditions change (e.g., pass/fail)
- Many errors occur at the edges of valid ranges

(Any two valid reasons accepted)

Section B: Test Plan Completion (14 marks)

96. Password test plan: (8 marks — 1 per cell, 2 for Type column)

Test #	Description	Input Data	Expected Output	Type
1	Valid password	"computer123"	"Password accepted"	Normal
2	Minimum length	"abcdefgh"	"Password accepted"	Boundary
3	Empty password	""	"Too short"	Erroneous
4	7 characters	"abcdefg"	"Too short"	Boundary
5	Very long password	"a" * 100	"Password accepted"	Normal

5. Day number test plan: (6 marks — 1 per correct cell)

Test #	Description	Input	Expected Output	Type
1	First day	1	"Monday"	Boundary
2	Middle day	4	"Thursday"	Normal
3	Last day	7	"Sunday"	Boundary
4	Too low	0	"Invalid day"	Erroneous
5	Too high	8	"Invalid day"	Erroneous
6	Decimal	3.5	"Invalid day"	Erroneous
7	Zero	0	"Invalid day"	Erroneous/
				Boundary

Section C: Design Your Own Test Plan (15 marks)

Model answer: (Any valid test plan with required coverage accepted)

Test #	Description	Input	Expected Output	Type
1	Normal mid-range	75	B	Normal
2	Normal pass	55	D	Normal

3	A*/A boundary	89	A	Boundary	
4	A/B boundary	79	B	Boundary	
5	B/C boundary	69	C	Boundary	
6	D/F boundary	49	F	Boundary	
7	Maximum	100	A*	Boundary	
8	Minimum	0	F	Boundary	
9	Negative mark	-10	Invalid mark	Erroneous	
10	Over 100	105	Invalid mark	Erroneous	
11	Non-numeric	"hi"	Invalid mark	Erroneous	

Marking: (15 marks)

- 2 normal tests with valid input and expected grade: 4 marks
- 4+ boundary tests at grade boundaries: 6 marks (1-2 per test)
- 3+ erroneous tests: 3 marks (1 per test)
- All tests have correct expected output: 2 marks

WEEK 16: PROGRAM DEVELOPMENT LIFECYCLE

LESSON 16.1 — PROGRAM DEVELOPMENT STAGES

Topic: Analysis, design, coding, testing, maintenance, documentation

Duration: 40 minutes

Lesson Type: Direct Instruction + Practical

Syllabus Ref: CIE 0478 Section 2.1 (Problem-Solving), 2.4 (Documentation)

Learning Objective:

By the end of this lesson, students will be able to describe the six stages of the program development lifecycle and apply analysis and design to a given problem specification.

Key Vocabulary:

- program development lifecycle (程序开发生命周期 / cheng xu kai fa sheng ming zhou qi)
- analysis (分析 / fen xi) — understanding the problem
- design (设计 / she ji) — planning the solution
- coding (编码 / bian ma) — writing the program
- testing (测试 / ce shi) — finding and fixing errors
- maintenance (维护 / wei hu) — updating after release
- documentation (文档 / wen dang) — explaining the program
- problem specification (问题规范 / wen ti gui fan)
- requirements (需求 / xu qiu)

- inputs and outputs (输入和输出 / shu ru he shu chu)
- pseudocode (伪代码 / wei dai ma)
- flowchart (流程图 / liu cheng tu)
- IPO chart (输入-处理-输出表 / shu ru chu li shu chu biao)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

Display: "Think of building a house. What steps would you take BEFORE you start building? List at least 4 steps."

Students write: planning, getting permission, designing, choosing materials, etc.

Teacher draws parallel: "Programming is the same — you don't just start typing code!"

3-12 min | DIRECT INSTRUCTION

Teacher explains the Program Development Lifecycle:

STAGE 1: ANALYSIS (What problem are we solving?)

- Read the problem specification carefully
- Identify: INPUTS (what data goes in?), OUTPUTS (what must come out?),

PROCESSING (what calculations/decisions are needed?),

CONSTRAINTS (any limits or special rules?)

- Write an IPO (Input-Process-Output) chart

STAGE 2: DESIGN (How will we solve it?)

- Write pseudocode — structured English describing the algorithm
- Draw a flowchart — visual representation of the logic
- Break complex problems into sub-problems (decomposition/modular design)

STAGE 3: CODING (Write the program)

- Convert pseudocode/flowchart into Python
- Follow good coding conventions (meaningful variable names, consistent indentation, comments)

STAGE 4: TESTING (Does it work correctly?)

- Use test plans with normal, boundary, and erroneous data
- Dry runs and trace tables
- Fix any errors found

STAGE 5: MAINTENANCE (Keep it working)

- Corrective: Fix bugs discovered after release
- Adaptive: Update for new requirements (e.g., new OS)

- Perfective: Improve performance or add features

STAGE 6: DOCUMENTATION (Explain it to others)

- Internal: Comments in the code
- External: User guide and technical guide

Visual diagram on board: CYCLE showing all 6 stages connected

in a circle (Analysis → Design → Code → Test → Maintain → Document
→ back to Analysis)

12-25 min | GUIDED PRACTICE

Problem specification:

"A teacher needs a program to calculate the average mark for their class. The program should:

- Ask how many students are in the class
- Ask for each student's mark (0-100)
- Calculate and display the class average
- Display how many students passed (mark ≥ 50)"

Task 1 (all): Complete the IPO chart:

Input	Process	Output
numStudents	Loop to get each mark	Class average
each mark	Calculate total and average	Number of passes
	Count marks ≥ 50	

Task 2 (all): Write pseudocode for this problem.

Scaffold for Emerging:

// Calculate class average and pass count

OUTPUT "Enter number of students: "

INPUT numStudents

total ← _____

passCount ← _____

FOR _____ FROM 1 TO numStudents DO

 OUTPUT "Enter mark for student " + _____ + ": "

 INPUT mark

 total ← _____ + _____

 IF _____ ≥ 50 THEN

 passCount ← _____ + 1

 END IF

END FOR

average ← _____ / _____

OUTPUT "Average mark: " + _____

OUTPUT "Students who passed: " + _____

25-35 min | **INDEPENDENT PRACTICE**

New problem specification:

"A shop needs a program for a simple checkout. The program should:

- Ask how many items the customer is buying
- For each item, ask for the price
- Calculate the total cost
- If total > \$100, apply 10% discount
- Display the final amount to pay"

Task: Write the complete IPO chart AND pseudocode for this problem.

Extension: Draw a flowchart for the discount decision.

35-40 min | **PLENARY + HOMEWORK**

Plenary: Show a strong pseudocode example on board. Highlight:

97. Clear variable names

98. Proper structure (sequence, selection, iteration)

99. Comments explaining each section

Homework: Worksheet 16A — Analysis and design practice.

EAL Support:

- Visual diagram of the 6-stage lifecycle on board with icons:

Analysis (magnifying glass), Design (blueprint), Code (keyboard),

Test (checkmark), Maintain (wrench), Document (paper)

- IPO chart template provided with bilingual headers

- Sentence frame for analysis: "The program needs ____ as input, will ____ as processing, and should output ____"

- Colour-coded stages on lifecycle diagram

- Glossary of stage names with Chinese translations

Differentiation:

Emerging: Complete IPO chart with teacher guidance. Fill in scaffolded pseudocode. Focus on understanding the 6 stages by name and order. Draw arrows connecting stages on lifecycle diagram.

Developing: Complete all tasks independently. Write full pseudocode without scaffolding. Add comments to pseudocode.

Secure: Complete all tasks + extension. Write pseudocode using functions/modules for decomposition. Research and explain why maintenance is the longest (and most expensive) stage in real software. Compare the lifecycle to the engineering design process.

Resources Needed:

- Projector + board
- Lifecycle diagram (large, for wall)
- IPO chart templates
- Scaffolded pseudocode handouts (Emerging)
- Worksheet 16A

--- LESSON 16.2 ---

LESSON 16.2 — DOCUMENTATION AND DEBUGGING

Topic: Internal/external documentation; debugging strategies; common Python errors

Duration: 40 minutes

Lesson Type: Direct Instruction + Practical

Syllabus Ref: CIE 0478 Section 2.4 (Documentation), 2.3 (Testing/Debugging)

Learning Objective:

By the end of this lesson, students will be able to write effective comments in code, explain the difference between internal and external documentation, use print statements and Thonny breakpoints for debugging, and identify common Python errors and their fixes.

Key Vocabulary:

- internal documentation (内部文档 / nei bu wen dang) — comments in code
- external documentation (外部文档 / wai bu wen dang) — user/technical guides
- comment (注释 / zhu shi)
- user guide (用户指南 / yong hu zhi nan)
- technical guide (技术指南 / ji shu zhi nan)
- debugging (调试 / tiao shi)
- breakpoint (断点 / duan dian)
- print statement debugging (打印调试 / da yin tiao shi)
- step through (逐步执行 / zhu bu zhi xing)
- syntax highlighting (语法高亮 / yu fa gao liang)
- error message (错误信息 / cuo wu xin xi)
- stack trace (堆栈跟踪 / dui zhan gen zong)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

Display two versions of the same code:

Version A (no comments):

```
def calc(x, y):
    return x + y / 2
```

Version B (with comments):

```
# calc_average: calculates the average of two numbers
# Parameters: num1, num2 — the two numbers to average
# Returns: the average value

def calc_average(num1, num2):
    return (num1 + num2) / 2
```

"Which version is better and why?" Students discuss in pairs.

3-12 min | DIRECT INSTRUCTION

Teacher explains:

INTERNAL DOCUMENTATION (Comments in code):

- Single-line comment: starts with #
- Multi-line comment: triple quotes """ ... """
- Good comments explain WHY, not WHAT. The code shows WHAT;

comments should explain intent.

- Bad comment: `x = x + 1 # Add 1 to x (obvious!)`
- Good comment: `x = x + 1 # Move to next position in array`
- Write comments for: function purpose, complex logic,

non-obvious decisions, and to mark sections

EXTERNAL DOCUMENTATION:

- User Guide: For END USERS. Explains how to USE the program.

Includes: installation, starting the program, what inputs to enter, what outputs mean, troubleshooting common problems.

- Technical Guide: For PROGRAMMERS. Explains how the code works.

Includes: system requirements, file structure, function descriptions, data structures used, algorithms explained.

DEBUGGING STRATEGIES:

100. Print statement debugging: Insert `print()` at key points to show variable values. Remove when fixed.
101. Thonny breakpoints: Click line number to set a red dot. Run with Debug (F5). Use Step Over (F6) to execute line by line. Watch the Variables pane to see values change.
102. Read the error message carefully! The LAST line tells you the error type. The arrow ^ shows WHERE the error is.

COMMON PYTHON ERRORS AND FIXES:

- `SyntaxError`: Check for missing colons, brackets, quotes
- `IndentationError`: Check spaces/tabs, ensure consistent indent

- `NameError`: Variable not defined — check spelling and scope
- `TypeError`: Wrong type used — check data type conversions
- `IndexError`: List index out of range — check loop bounds
- `ValueError`: Wrong value — check input conversion (`int()`)
- `ZeroDivisionError`: Can't divide by zero — check denominator
- `FileNotFoundError`: File doesn't exist — check path and filename

Live demo: Teacher introduces a bug in a known-working program and uses Thonny's debugger to find it.

12-25 min | **GUIDED PRACTICE**

Task 1 (all): Given a working program (student calculator), add comments to every significant line explaining what it does and why.

Task 2 (all): Teacher provides a program with 5 deliberate errors. Students find and fix each error, documenting what the error was and how they fixed it.

Buggy program:

```
def calculate_average(scores)
    total = 0
    for i in range(scores)
        total = total + scores[i]
    average = total / len(scores)
    return average
marks = [85, 90, 78, 92]
result = calculate_averge(marks)
print("The average is" result)
(Errors: missing colon on def, missing colon on for, missing []
around len, misspelled function name, missing comma in print)
```

25-35 min | **INDEPENDENT PRACTICE**

Task 1: Write a short User Guide (5-8 bullet points) for the student management system from Week 14.

Task 2: Use Thonny's debugger (breakpoints + step over) to trace through a factorial program. Write down the value of 'n' and 'factorial' at each breakpoint.

Extension: Write a Technical Guide for a given program, including:

- What the program does
- What files it uses
- What functions are defined and what they do
- What data structures are used

35-40 min | **PLENARY + HOMEWORK**

Plenary: Share one debugging tip with the class. Create a class

"Debugging Toolkit" poster with all tips collected.

Key question: "Why is documentation important if the code already works?" (Answer: others need to read/maintain it; you will forget your own code after a few weeks!)

Homework: Worksheet 16B — Documentation and debugging.

EAL Support:

- Comment symbol # displayed large on board with "注释" label
- Visual guide: "Good comment vs Bad comment" comparison on board
- Thonny debugger screenshot with labels in English and Chinese
- Common errors cheat sheet with Chinese translations
- Sentence frame for comments: "This function ____ because ____"

Differentiation:

Emerging: Focus on adding simple comments (what each line does). Use the error-finding activity with hints (number of errors per section given). Write a 3-point user guide.

Developing: Add meaningful comments (explaining why, not just what). Find all 5 errors independently. Write a complete user guide.

Secure: Add professional-level comments. Fix errors AND improve the code. Write both user guide and technical guide. Use Thonny debugger to trace a complex program and explain the call stack. Create a "Debugging Flowchart" that helps others decide which strategy to use based on the error type.

Resources Needed:

- Projector + Thonny with debugger demo
- Buggy program printouts
- Thonny debugger guide handout
- User guide template
- Worksheet 16B

--- WORKSHEET 16A ---

WORKSHEET 16A — PROGRAM DEVELOPMENT: ANALYSIS AND DESIGN

Name: _____ Class: _____ Date: _____

Section A: Lifecycle Knowledge (8 marks)

103. List the SIX stages of the program development lifecycle in order. (6 marks)

104. _____

105. _____

106. _____
 107. _____
 108. _____
 109. _____

110. Which stage is usually the LONGEST in a real software project? (1 mark)
 111. In which stage would you write pseudocode and draw flowcharts? (1 mark)

Section B: Analysis — IPO Chart (10 marks)

4. Complete the IPO chart for this problem:

"A program asks for the length and width of a rectangle, calculates the area and perimeter, and displays both results."

Inputs	Processing	Outputs
--------	------------	---------

5. Complete the IPO chart for this problem:

"A program reads a list of temperatures and finds the highest and lowest values."

Inputs	Processing	Outputs
--------	------------	---------

Section C: Design — Pseudocode (12 marks)

112. Write pseudocode for the rectangle problem (Question 4). Include comments.

7. Write pseudocode for this problem:

"A program asks the user to enter 5 exam marks. It calculates the average and prints whether the student passed (average ≥ 50) or failed."

Section D: Matching (5 marks)

8. Match each description to the correct stage.

- a) Writing Python code in Thonny ____ Analysis
 b) Fixing a bug reported by a user ____ Design
 c) Drawing a flowchart of the algorithm ____ Coding
 d) Reading the problem specification ____ Testing
 e) Creating a test plan ____ Maintenance

TOTAL: 35 marks

--- WORKSHEET 16B ---

WORKSHEET 16B — DOCUMENTATION AND DEBUGGING

Name: _____ Class: _____ Date: _____

Section A: Documentation Types (8 marks)

113. What is the difference between internal and external documentation? (2 marks)

2. Give two examples of internal documentation: (2 marks)

a) _____

b) _____

3. Give two examples of external documentation: (2 marks)

a) _____

b) _____

114. Is this a good comment? Explain why or why not: (2 marks)

```
x = x + 1 # increase x by 1
```

Section B: Add Comments (8 marks)

5. Add meaningful comments to the following code:

```
---  
  
def calculate_bmi(weight, height):  
    bmi = weight / (height * height)  
    if bmi < 18.5:  
        return "Underweight"  
    elif bmi < 25:  
        return "Normal"  
    elif bmi < 30:  
        return "Overweight"  
    else:  
        return "Obese"  
  
user_weight = float(input("Weight (kg): "))  
user_height = float(input("Height (m): "))  
result = calculate_bmi(user_weight, user_height)  
print("Your BMI category: " + result)  
  
---
```

Section C: Error Finding and Fixing (10 marks)

115. The following program has 5 errors. Find each one, state the error type, and write the corrected line.

Program: Read 3 numbers and find the largest

```
def find_largest(nums)
```

```

largest = 0
for i in range(len(nums))
    if nums[i] > largest
        largest = nums[i]
return largest
numbers = [45, 92, 38]
answer = find_largest(numbers)
print("Largest: " + answer)

```

Line	Error Description	Error Type	Corrected Line
------	-------------------	------------	----------------

Section D: Debugging Strategy (9 marks)

116. A program crashes with: `IndexError: list index out of range`

- What does this error mean? (2 marks)
- Name TWO strategies you could use to find the problem: (2 marks)
- What should you check in your code? (2 marks)

117. Explain how you would use Thonny's debugger to find a logic error. (3 marks)

TOTAL: 35 marks

--- ANSWER KEY (Week 16) ---

WORKSHEET 16A — ANSWER KEY

Section A: Lifecycle Knowledge (8 marks)

- (6 marks — 1 each)
- Analysis
- Design
- Coding
- Testing
- Maintenance
- Documentation

(Must be in this order)

125. Maintenance (1 mark) — Most software spends years being maintained after initial development.

3. Design (1 mark)

Section B: Analysis — IPO Chart (10 marks)

4. Rectangle problem: (5 marks)

Inputs	Processing	Outputs
length	Calculate area = l * w	area

| width | Calculate perimeter = 2(l+w) | perimeter |
(2 marks for inputs, 2 for processing, 1 for outputs)

5. Temperature problem: (5 marks)

Inputs	Processing	Outputs
list of	Initialise highest/lowest	highest temp
temperatures	Loop through list	lowest temp
	Compare and update max/min	
(2 marks for inputs, 2 for processing, 1 for outputs)

Section C: Design — Pseudocode (12 marks)

6. Rectangle pseudocode — model answer: (6 marks)

```
---  
// Calculate area and perimeter of rectangle  
OUTPUT "Enter length: "  
INPUT length  
OUTPUT "Enter width: "  
INPUT width  
area ← length * width                      (1 mark)  
perimeter ← 2 * (length + width)          (1 mark)  
OUTPUT "Area: " + area                    (1 mark)  
OUTPUT "Perimeter: " + perimeter          (1 mark)  
// Comments and structure                  (2 marks)  
---
```

7. Exam marks pseudocode — model answer: (6 marks)

```
---  
// Check if student passed based on 5 exam marks  
OUTPUT "Enter 5 exam marks: "            (1 mark)  
total ← 0  
FOR i FROM 1 TO 5 DO                      (1 mark)  
    INPUT mark  
    total ← total + mark  
END FOR  
average ← total / 5                        (1 mark)  
OUTPUT "Average: " + average              (1 mark)
```

IF average >= 50 THEN (1 mark)

 OUTPUT "Passed"

ELSE

 OUTPUT "Failed"

END IF (1 mark)

Section D: Matching (5 marks)

126. a) Coding, b) Maintenance, c) Design, d) Analysis, e) Testing

WORKSHEET 16B — ANSWER KEY

Section A: Documentation Types (8 marks)

127. Internal documentation is written inside the program code (comments).

External documentation is separate from the code (user guides,
technical manuals). (2 marks)

2. Any two of: (2 marks — 1 each)

- Single-line comments (#)
- Multi-line comments (""" ... """)
- Docstrings
- Inline comments explaining complex logic

3. Any two of: (2 marks — 1 each)

- User guide / User manual
- Technical guide / Technical documentation
- Installation guide
- README file
- API documentation

128. This is NOT a good comment. (1 mark) The comment just restates what the code obviously does. A good comment would explain WHY we are incrementing

x, e.g., "Move to next array index" or "Skip to next record." (1 mark)

Section B: Add Comments (8 marks)

Model comments (accept any meaningful comments):

Function to calculate BMI category from weight and height

```
def calculate_bmi(weight, height):
```

```
    # Calculate BMI using formula: weight / height^2
```

```

bmi = weight / (height * height)
# Determine BMI category based on WHO standards
if bmi < 18.5:
    return "Underweight" # Below healthy range
elif bmi < 25:
    return "Normal"      # Healthy weight range
elif bmi < 30:
    return "Overweight"  # Above healthy range
else:
    return "Obese"       # Significantly above healthy range

```

Get user input for weight in kilograms

```
user_weight = float(input("Weight (kg): "))
```

Get user input for height in metres

```
user_height = float(input("Height (m): "))
```

Call function and store result

```
result = calculate_bmi(user_weight, user_height)
```

Display the BMI category to the user

```
print("Your BMI category: " + result)
```

(1 mark per meaningful comment, up to 8 marks)

Section C: Error Finding and Fixing (10 marks)

Error #	Description	Type	Corrected
1	Missing colon after params	Syntax	def find_largest(nums):
2	Missing colon after range()	Syntax	for i in range(len(nums)):
3	Missing colon after if	Syntax	if nums[i] > largest:
4	Missing closing bracket	Syntax	answer = find_largest(numbers)
5	Cannot concatenate str+int	Runtime	print("Largest: " + str(answer))

(2 marks per error — 1 for identification, 1 for correction)

Section D: Debugging Strategy (9 marks)

7a. This error means the program tried to access an element in a list using an index that doesn't exist (e.g., index 5 in a 3-element list). (2 marks)

7b. Any two valid strategies: (2 marks — 1 each)

- Add print statements to show index value and list length
- Use Thonny debugger with breakpoints
- Check loop conditions
- Trace through manually with a trace table

7c. Check: (2 marks)

- Loop bounds (range values)
- List length vs index being accessed
- Whether the list is empty
- Off-by-one errors (using $\leq$ instead of $<$)

8. Using Thonny debugger: (3 marks)

- Set a breakpoint by clicking the line number (red dot appears)
- Run the program in debug mode (Ctrl+F5 or Debug button)
- Use "Step Over" (F6) to execute one line at a time
- Watch the Variables pane to see values change after each line
- Identify where the actual value differs from expected value

(1 mark per step, any 3 valid steps accepted)

WEEK 17: REVISION CLINIC

LESSON 17.1 — MIXED REVISION: ALGORITHMS, PSEUDOCODE & FLOWCHARTS

Topic: Station-based revision covering algorithms, pseudocode, flowcharts, and trace tables from the entire year.

Duration: 40 minutes

Lesson Type: Stations/Rotation Activity

Syllabus Ref: CIE 0478 Section 1.1-1.3, 2.1-2.3 (All prior topics)

Learning Objective:

By the end of this lesson, students will be able to apply their knowledge of algorithms, pseudocode, flowcharts, and trace tables through a series of practical revision stations.

Key Vocabulary:

- revision (复习 / fu xi)
- station activity (站点活动 / zhan dian huo dong)
- algorithm (算法 / suan fa)
- pseudocode (伪代码 / wei dai ma)
- flowchart (流程图 / liu cheng tu)
- trace table (追踪表 / zhui zong biao)
- flowchart symbols (流程图符号 / liu cheng tu fu hao)
- sequence (顺序 / shun xu)

- selection (选择 / xuan ze)
- iteration (循环 / xun huan)
- speed challenge (速度挑战 / su du tiao zhan)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

"Write down 3 things you remember about pseudocode and 3 things you remember about flowcharts. You have 2 minutes — go!"

Teacher collects key points on board as starter.

3-5 min | BRIEFING

Teacher explains the 4 stations. Students will rotate through stations spending ~8 minutes at each. Each station has a challenge to complete. Teacher assigns starting stations (pre-planned groups).

5-37 min | STATIONS ACTIVITY (8 minutes per station, 1 minute transition)

STATION 1: Pseudocode Challenge (桌 1 — 伪代码站)

Task: Write pseudocode for these problems:

- Beginner: Ask user for a number, print whether it is positive, negative, or zero.
- Intermediate: Ask for 10 numbers, count how many are even.
- Advanced: Read student marks from a file, calculate average, print grade (A/B/C/D/F).

Materials: Pseudocode reference sheet, lined paper

STATION 2: Flowchart Station (桌 2 — 流程图站)

Task: Draw flowcharts for:

- Beginner: Input a password. If password = "secret", print "Access granted", else print "Access denied".
- Intermediate: A WHILE loop that counts from 1 to 10.
- Advanced: A program that reads numbers until the user enters -1, then prints the largest number entered.

Materials: Flowchart symbols reference, A3 paper, markers

STATION 3: Trace Table Station (桌 3 — 追踪表站)

Task: Complete trace tables for:

- Beginner: Simple FOR loop adding numbers
- Intermediate: WHILE loop with nested IF
- Advanced: Program with 2D array processing

Materials: Pre-printed trace table templates, calculator

STATION 4: Mixed Problems Station (桌 4 — 综合题站)

Task: Exam-style questions:

- Explain the difference between linear and binary search
- State one advantage and one disadvantage of arrays vs files
- Write Python code to swap two variables
- What is the output of this code? (given code snippet)

Materials: Question cards, answer sheets

Teacher circulates, supports struggling students, keeps time.

37-40 min | **PLENARY + HOMEWORK**

Quick review: "What was the hardest station? What do you need to revise more?" Students write one strength and one target on a post-it, stick on "Progress Wall".

Homework: Complete any unfinished station work + Worksheet 17A.

EAL Support:

- Station signs in both English and Chinese
- Vocabulary cards at each station with key terms
- Pair EAL students with supportive peers
- Visual flowchart symbol reference at Station 2
- Sentence starters for explanations at Station 4
- Allow EAL students to answer in Chinese at mixed problems station if needed

Differentiation:

Emerging: Start at Station 1 (Beginner tasks). Teacher stays nearby.

Provide heavily scaffolded templates. Focus on completing at least 2 stations fully rather than rushing through all 4. Peer support from developing/secure students at same station.

Developing: Complete all stations at intermediate level. Attempt advanced tasks if time allows. Support emerging students at same station.

Secure: Complete all advanced tasks. Act as "station helper" — assist peers after completing own work. Create an additional challenge question for your weakest station.

Resources Needed:

- 4 station setups around the room with signs
- Station task cards (laminated, reusable)
- Pseudocode reference sheets
- Flowchart symbol reference sheets
- Trace table templates
- A3 paper and markers (Station 2)
- Post-it notes for plenary

- Timer (visible to class)
- Worksheet 17A (homework)

--- LESSON 17.2 ---

LESSON 17.2 — MIXED REVISION: PYTHON PROGRAMMING

Topic: Practical coding challenges covering all Year 9 Python topics

Duration: 40 minutes

Lesson Type: Practical Coding

Syllabus Ref: CIE 0478 Section 2.2 (Programming - all subsections)

Learning Objective:

By the end of this lesson, students will be able to write a complete Python program that combines file handling, loops, selection, and calculations, and complete a self-assessment checklist of their programming skills.

Key Vocabulary:

- coding challenge (编程挑战 / bian cheng tiao zhan)
- self-assessment (自我评估 / zi wo ping gu)
- combine (组合 / zu he)
- checklist (检查清单 / jian cha qing dan)
- exam topic (考试主题 / kao shi zhu ti)
- recap (回顾 / hui gu)
- skill area (技能领域 / ji neng ling yu)
- target area (目标领域 / mu biao ling yu)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

Display on board: "Write a Python function called greeting(name) that returns 'Hello, [name]!'." Quick recall of function syntax.

Students type in Thonny. Teacher spot-checks.

3-8 min | DIRECT INSTRUCTION — TOPIC RECAP

Teacher rapidly reviews key topics using "One Slide Per Topic":

- Variables & I/O: input(), print(), int(), float(), str()
- Selection: if/elif/else, comparison operators
- Iteration: for, while, range(), nested loops
- Arrays: 1D lists, 2D lists, indexing, len()
- Strings: indexing, slicing, .split(), .strip(), len()
- Functions: def, parameters, return, scope
- File Handling: open(), read/write, with statement, CSV
- Testing: error types, trace tables, test plans

Each slide has ONE key code example. Students follow along.

8-30 min | PRACTICAL CODING CHALLENGE

Main Task: Write a complete Python program that:

"A school wants to analyse exam results. Write a program that:

129. Reads a CSV file 'results.csv' containing student names and marks (format: Name,Math,Science,English)
130. Calculates each student's average mark
131. Assigns a grade: A* (90+), A (80-89), B (70-79), C (60-69), D (50-59), F (below 50)
132. Writes the results to a new file 'grades.csv' with format: Name,Average,Grade
133. Prints a summary showing: total students, class average, and how many of each grade were awarded"

This is a SYNTHESIS task — combines file reading, file writing, loops, selection, string manipulation, and calculations.

Scaffolding for Emerging (provided as handout):

Step 1: Open the file for reading

with open(_____, _____) as infile:

 outfile = open(_____, _____)

 outfile.write(_____ + "\n") # Header

 total_all = 0

 count = 0

 # For counting grades

 a_star = 0; a = 0; b = 0; c = 0; d = 0; f = 0

 header = infile.readline() # Skip header

 for _____ in infile:

 data = _____.strip().split(_____)

 name = _____

 math = _____

 science = _____

 english = _____

 average = _____

 total_all = total_all + average

 count = count + 1

 # Determine grade

 if _____ >= 90:

 grade = "A*"

```

        a_star = a_star + 1
    elif _____ >= 80:
        grade = _____
        a = a + 1

# ... continue for other grades

outfile.write(name + "," + str(average) + "," + grade + "\n")

class_avg = _____

outfile.close()

# Print summary

print(_____)

```

30-37 min | SELF-ASSESSMENT CHECKLIST

Students complete the self-assessment checklist:

PROGRAMMING SKILLS CHECKLIST — Year 9 Computer Science

Skill Area	Confident	Okay	Need Help
Variables and data types	☐	☐	☐
Input and output	☐	☐	☐
If statements (selection)	☐	☐	☐
For loops	☐	☐	☐
While loops	☐	☐	☐
1D arrays (lists)	☐	☐	☐
2D arrays	☐	☐	☐
String manipulation	☐	☐	☐
Functions	☐	☐	☐
Scope (local/global)	☐	☐	☐
Reading from files	☐	☐	☐
Writing to files	☐	☐	☐
CSV processing	☐	☐	☐
Error handling (try/except)	☐	☐	☐
Trace tables	☐	☐	☐
Test plans	☐	☐	☐
Pseudocode	☐	☐	☐
Flowcharts	☐	☐	☐

My top 3 strengths:

134. _____
135. _____
136. _____

My top 3 areas to improve:

137. _____

138. _____

139. _____

37-40 min | PLENARY + HOMEWORK

"Look at your checklist. Choose ONE skill you marked 'Need Help' and tell your partner one thing you will do to improve it."

Whole class: "What was the hardest part of the coding challenge?"

Common answers: file handling, grade boundary logic, or counting grades. Teacher addresses most common issues.

Homework: Worksheet 17B — Revision questions + prepare for mock exam.

EAL Support:

- Bilingual checklist with Chinese skill names
- Sentence starters for self-assessment: "I am confident with ____ because..." and "I need help with ____ because..."
- Visual emoji scale on checklist: Confident / Okay / Need Help
- Pair discussion in mixed language groups
- Key Python keywords reference sheet available

Differentiation:

Emerging: Complete the coding challenge with scaffolded code (fill blanks).

Focus on getting the file reading and basic loop working.

Complete checklist with teacher assistance. Target: 50% of scaffolded code correct.

Developing: Complete coding challenge independently. Use scaffold only if stuck after 5 minutes. Complete full checklist. Target: Working program with correct output.

Secure: Complete coding challenge + add enhancements: input validation, formatted output, error handling for missing files. Complete checklist and set specific revision targets for mock exam.

Target: Fully working program with all features + extensions.

Support peers who are struggling.

Resources Needed:

- Projector + Thonny
- results.csv sample file
- Scaffolded code handouts (Emerging)
- Self-assessment checklist (printed)

- Python keyword reference sheet

- Worksheet 17B

--- WORKSHEET 17A ---

WORKSHEET 17A — MIXED REVISION: ALGORITHMS AND PSEUDOCODE

Name: _____ Class: _____ Date: _____

Section A: Pseudocode Writing (12 marks)

140. Write pseudocode for a program that asks the user to enter 5 numbers and prints the largest one.

141. Write pseudocode for a program that reads names from a file and prints only those that start with 'S'.

Section B: Trace Table (10 marks)

3. Complete the trace table for this algorithm:

$x \leftarrow 3$

$y \leftarrow 8$

WHILE $x < y$ DO

$x \leftarrow x + 2$

$y \leftarrow y - 1$

OUTPUT $x + ", " + y$

END WHILE

Iteration	x	y	$x < y?$	Output
Initial			-	-
1				
End				

Section C: Flowchart Symbols (5 marks)

4. Draw the correct flowchart symbol for each:

a) Start/End: _____

b) Process/Action: _____

c) Decision: _____

d) Input/Output: _____

e) Arrow/Flow line: _____

Section D: Search Algorithms (8 marks)

142. State TWO requirements for binary search to work: (2 marks)

a) _____

b) _____

143. A list has 1000 elements. What is the maximum number of comparisons needed for: (2 marks)

a) Linear search: _____

b) Binary search: _____

7. What is the time complexity (Big O) of: (2 marks)

a) Linear search: _____

b) Binary search: _____

144. In what situation would linear search be BETTER than binary search? (2 marks)

TOTAL: 35 marks

--- WORKSHEET 17B ---

WORKSHEET 17B — REVISION: PYTHON PROGRAMMING AND FILE HANDLING

Name: _____ Class: _____ Date: _____

Section A: Code Reading (10 marks)

Read the following program and answer the questions.

```
def process_scores(filename):
    try:
        with open(filename, "r") as file:
            file.readline() # Skip header
            highest = -1
            best = ""
            total = 0
            count = 0
            for line in file:
                data = line.strip().split(",")
                name = data[0]
                score = int(data[1])
                total = total + score
                count = count + 1
                if score > highest:
                    highest = score
                    best = name
            average = total / count
```

```

        return best, highest, average
    except FileNotFoundError:
        return "Error", 0, 0
winner, top_score, avg = process_scores("scores.csv")
print("Winner: " + winner)
print("Top score: " + str(top_score))
print("Average: " + str(avg))
---
```

145. What does this function return if the file is not found? (2 marks)
146. What is the initial value of highest and why is it set to -1? (2 marks)
147. What would happen if we forgot int() around data[1]? (2 marks)

4. The file "scores.csv" contains:

```

Name,Score
Amy,78
Ben,92
Cat,65
Dan,88
```

Write the EXACT output of the program: (4 marks)

Section B: Programming Challenge (15 marks)

5. Write a complete Python program that:

- a) Creates a list of 5 student names (2 marks)
- b) Asks the user to enter a mark for each student (3 marks)
- c) Stores the marks in a parallel list (2 marks)
- d) Calculates and prints the class average (3 marks)
- e) Prints each student's name, mark, and whether they passed (≥ 50) (5 marks)

Section C: Error Spotting (10 marks)

148. Each of the following code snippets contains an error. Identify the error type (syntax, logic, runtime) and write the corrected code.

- a) (2 marks)

```

if x > 10
    print("Big")
```

Error: _____ Correction: _____

- b) (2 marks)

```

numbers = [4, 7, 2]
```



```
print(numbers[3])
```

Error: _____ Correction: _____

c) (2 marks)

```
total = 0
```

```
for i in range(5):
```

```
    total = total + i
```

```
print(total / 5)
```

Expected: average of 1,2,3,4,5 = 3.0 but output is different

Error: _____ Correction: _____

d) (2 marks)

```
def greet(name)
```

```
    return "Hello " + name
```

Error: _____ Correction: _____

e) (2 marks)

```
age = input("Enter age: ")
```

```
if age > 18:
```

```
    print("Adult")
```

Error: _____ Correction: _____

TOTAL: 35 marks

--- ANSWER KEY (Week 17) ---

WORKSHEET 17A — ANSWER KEY

Section A: Pseudocode Writing (12 marks)

1. Find largest of 5 numbers — model answer: (6 marks)

```
// Find the largest of 5 numbers
```

```
largest ← 0 (1 mark)
```

```
FOR i FROM 1 TO 5 DO (1 mark)
```

```
    OUTPUT "Enter number " + i + ": "
```

```
    INPUT number
```

```
    IF i == 1 OR number > largest THEN (2 marks)
```

```
        largest ← number
```

```
    END IF
```

```
END FOR
```

```
OUTPUT "The largest number is: " + largest (1 mark)
```

```
// Structure and comments (1 mark)
```

(Alternative: set largest to first input, then compare)

2. Names starting with 'S' — model answer: (6 marks)

```
// Read names and print those starting with S
OPEN "names.txt" FOR READ          (1 mark)
FOR EACH line IN file DO
    name ← line.strip()            (1 mark)
    IF name[0] == "S" THEN          (2 marks)
        OUTPUT name                (1 mark)
    END IF
END FOR
CLOSE file                          (1 mark)
```

Section B: Trace Table (10 marks)

3. Completed trace table:

Iteration	x	y	x < y?	Output
Initial	3	8	-	-
1	3	8	True	5, 7
After	5	7	True	-
2	5	7	True	7, 6
After	7	6	False	-
End	7	6	-	-

(1 mark per row iterations 1-2, 2 marks for correct loop termination,
2 marks for correct outputs)

Section C: Flowchart Symbols (5 marks)

149. a) Oval/Rounded rectangle
 b) Rectangle
 c) Diamond/Rhombus
 d) Parallelogram
 e) Arrow

(1 mark each)

Section D: Search Algorithms (8 marks)

150. a) The data must be sorted (in order)
 b) The data must support random access (e.g., array, not linked list)

(1 mark each)

151. a) 1000 comparisons (worst case — check every element)

b) 10 comparisons ($\log_2(1000) \approx 9.97$, so maximum 10)

(1 mark each)

152. a) $O(n)$ b) $O(\log n)$ (1 mark each)

8. Linear search is better when: (2 marks)

- The data is not sorted (sorting takes extra time)
- The list is very small (overhead of binary search not worth it)
- You only need to search once (no benefit from sorting)
- You need to find ALL occurrences, not just one

(Any valid reason accepted)

WORKSHEET 17B — ANSWER KEY

Section A: Code Reading (10 marks)

1. Returns the tuple: ("Error", 0, 0) (2 marks)

153. highest is set to -1 because all valid scores are ≥ 0 . Starting at -1 ensures the first score will always be higher, so best gets a value.

(2 marks — 1 for value, 1 for explanation)

154. Without `int()`, `data[1]` is a string. The `+` operation with `total` (an `int`) would cause a `TypeError` (cannot add string and `int`). (2 marks)

4. Exact output: (4 marks — 1 per line, 1 for exact values)

Winner: Ben

Top score: 92

Average: 80.75

Section B: Programming Challenge (15 marks)

5. Model answer:

**Create lists for names and marks
(2 marks)**

```
names = ["Amy", "Ben", "Cat", "Dan", "Eve"]
```

```
marks = []
```

Ask for marks for each student (3 marks)

```
for i in range(len(names)):
    mark = int(input("Enter mark for " + names[i] + ": "))
    marks.append(mark)                (2 marks)
```

Calculate average (3 marks)

```
total = 0
for mark in marks:
    total = total + mark
average = total / len(marks)
print("Class average: " + str(average))
```

Print results with pass/fail (5 marks)

```
for i in range(len(names)):
    if marks[i] >= 50:
        result = "Pass"
    else:
        result = "Fail"
    print(names[i] + ": " + str(marks[i]) + " - " + result)
```

(Alternative valid approaches accepted)

Section C: Error Spotting (10 marks)

155. a) Syntax error — missing colon

Correction: if x > 10: (2 marks)

b) Runtime error — IndexError (index 3 doesn't exist, valid are 0,1,2)

Correction: print(numbers[2]) or change to loop (2 marks)

c) Logic error — range(5) gives 0,1,2,3,4 not 1,2,3,4,5

Correction: range(1, 6) or adjust total calculation (2 marks)

d) Syntax error — missing colon after function definition

Correction: def greet(name): (2 marks)

e) Runtime/Logic error — input returns string, cannot compare string to int

Correction: `age = int(input("Enter age: "))` (2 marks)

WEEK 18: MOCK EXAM & REVIEW

LESSON 18.1 — PAPER 2 STYLE MOCK EXAM

Topic: Full mock examination — Paper 2 style (Programming and Problem Solving)

Duration: 60 minutes (1 hour) + 5 min reading time

Lesson Type: Formal Assessment

Syllabus Ref: CIE 0478 Full Syllabus (Sections 1.1-1.3, 2.1-2.4)

Learning Objective:

Students will demonstrate their understanding of algorithms, pseudocode, Python programming, file handling, testing, and program development through a formal mock examination.

Instructions for Students:

- Time allowed: 60 minutes
- Total marks: 50
- Answer ALL questions
- Write your answers clearly in the spaces provided
- For pseudocode questions, use Cambridge-standard pseudocode
- For Python questions, ensure correct syntax
- Calculators are NOT permitted
- No access to Thonny or reference materials

YEAR 9 COMPUTER SCIENCE — MOCK EXAMINATION

Paper 2: Programming and Problem Solving

Time: 1 hour

Total Marks: 50

Student Name: _____

Class: _____ Date: _____

SECTION A — MULTIPLE CHOICE AND SHORT ANSWER (20 marks)

Answer ALL questions in this section.

Question 1 (1 mark)

Which Python function is used to convert a string to an integer?

A) `str()` B) `int()` C) `float()` D) `input()`

Answer: _____

Question 2 (1 mark)

What is the output of the following code?

```
for i in range(3):  
    print(i)
```

A) 1 2 3 B) 0 1 2 C) 0 1 2 3 D) 1 2

Answer: _____

Question 3 (1 mark)

Which of these is a SYNTAX error?

- A) A program that divides by zero
- B) A program that prints the wrong answer
- C) A program with a missing colon after an if statement
- D) A program that tries to open a missing file

Answer: _____

Question 4 (1 mark)

What is the purpose of try/except in Python?

- A) To make the program run faster
- B) To repeat a block of code
- C) To handle errors without crashing the program
- D) To define a function

Answer: _____

Question 5 (1 mark)

Which file mode opens a file for writing and DELETES any existing content?

A) 'r' B) 'w' C) 'a' D) 'x'

Answer: _____

Question 6 (2 marks)

State TWO requirements for binary search to work correctly.

Question 7 (2 marks)

What is the difference between a logic error and a runtime error?

Question 8 (3 marks)

Write Python code to open a file called "data.txt" for reading, read all lines into a list, and print the number of lines. Use a with statement.

Question 9 (2 marks)

A file contains the line: "Alice,85,92,78"

Write TWO lines of Python code to split this line and store Alice's name in a variable called 'name' and her first score in a variable called 'score1' (as an integer).

Question 10 (3 marks)

Complete the trace table for the following algorithm:

num $\leftarrow$ 1

sum $\leftarrow$ 0

WHILE num $\leq$ 4 DO

 sum $\leftarrow$ sum + num * num

 num $\leftarrow$ num + 1

END WHILE

OUTPUT sum

Iteration	num	sum	num $\leq$ 4?	Output
Initial	1	0	-	-
1				
2				
3				
4				
End			-	

Question 11 (3 marks)

State the purpose of each of these flowchart symbols:

a) Diamond/Rhombus: _____

b) Parallelogram: _____

c) Rectangle: _____

SECTION B — EXTENDED ANSWER (18 marks)

Answer ALL questions in this section.

Question 12 (6 marks)

A teacher wants a program to help with classroom seating.

(a) Write pseudocode for a program that:

- Asks the user to enter the number of rows in the classroom
- Asks the user to enter the number of seats per row
- Calculates the total number of seats
- If total seats ≥ 30 , print "Large classroom"
- Otherwise, print "Small classroom"

(b) Draw a flowchart for the decision "If total seats ≥ 30 " from part (a).

Use the correct symbols.

[Draw in the space below]

Question 13 (6 marks)

Write a complete Python program that:

- (a) Creates an empty list called 'tasks' (1 mark)
- (b) Uses a WHILE loop to repeatedly ask the user to enter a task
(or 'done' to finish) (2 marks)
- (c) Stores each task in the list (1 mark)
- (d) After the user enters 'done', writes all tasks to a file called
"tasks.txt", one per line (2 marks)

Question 14 (6 marks)

A program stores student exam results in a CSV file. Each line contains:

Name,Score. The file "results.csv" contains:

Name,Score

Amy,78

Ben,92

Cat,65

Dan,88

Eve,71

- (a) Write Python code that reads this file, calculates the average score,
and prints it. Use a with statement. (4 marks)
- (b) Extend your code to also find and print the name of the student with
the highest score. (2 marks)

SECTION C — TESTING AND DEVELOPMENT (12 marks)

Answer ALL questions in this section.

Question 15 (4 marks)

A program asks the user to enter their age (valid range: 0-120). The program should reject invalid ages.

Complete this test plan with one test of EACH type:

Test #	Description	Input Data	Expected Output	Test Type
1	Normal age	25	"Age accepted"	Normal
2				Boundary
3				Boundary
4				Erroneous

Question 16 (4 marks)

The following Python function has a logic error:

```
def calculate_average(numbers):  
    total = 0  
    for num in numbers:  
        total = total + num  
    return total    # BUG: should be total / len(numbers)
```

(a) Explain what the logic error is and what incorrect output it causes.

Use the input [10, 20, 30] in your explanation. (2 marks)

(b) Write the corrected line of code. (1 mark)

(c) Describe ONE strategy you could use to find this type of error. (1 mark)

Question 17 (4 marks)

Describe the stages of the program development lifecycle. For TWO stages, explain what happens and give a specific example related to creating a student attendance tracking program.

END OF PAPER

Total Marks: 50

--- LESSON 18.2 ---

LESSON 18.2 — MOCK EXAM REVIEW AND GAP ANALYSIS

Topic: Reviewing mock exam answers, identifying common errors, setting targets

Duration: 40 minutes

Lesson Type: Review and Reflection

Syllabus Ref: N/A — Review lesson

Learning Objective:

By the end of this lesson, students will be able to identify their own errors in the mock exam, understand the correct solutions, and set specific targets for Year 10 Computer Science.

Key Vocabulary:

- review (复习 / fu xi)
- gap analysis (差距分析 / cha ju fen xi)
- common error (常见错误 / chang jian cuo wu)
- target (目标 / mu biao)
- improvement (改进 / gai jin)
- mark scheme (评分标准 / ping fen biao zhun)
- self-reflection (自我反思 / zi wo fan si)
- next steps (下一步 / xia yi bu)

Lesson Structure (40 Minutes):

0-5 min | DO NOW — RETURN PAPERS

Distribute marked papers. Students silently review their marks and identify which sections they lost marks in. No discussion yet.

5-15 min | SECTION A REVIEW

Go through each Question 1-11 quickly. For each:

- State the correct answer
- Ask: "Who got this right?" (show of hands)
- If < 70% correct, explain the concept briefly

Key explanations to have ready:

- Q3: Syntax error = grammar mistake (won't run). Logic = runs but

wrong answer. Runtime = crashes during execution.

- Q5: 'w' overwrites, 'a' appends. This is a common mistake!
- Q10: Trace table = num goes 1,2,3,4,5. sum = 0+1+4+9+16 = 30.

Common error: forgetting num*num means square, or off-by-one.

15-30 min | SECTION B AND C REVIEW

Display model answers on board. Students mark their own papers using green pen (if not already marked).

Focus discussion on:

- Q12(a): Pseudocode structure marks (sequence, selection, I/O)
- Q13: Common error — forgetting 'done' is not saved to file
- Q14: Common error — forgetting to skip header row

- Q15: Boundary tests must be at the edges (0 and 120)
- Q16: Logic errors are hardest because the program runs!
- Q17: Two stages explained with specific examples

Peer marking protocol:

- Swap papers with partner
- Check against displayed mark scheme
- Write correct answers in green pen where wrong
- Return papers

30-37 min | GAP ANALYSIS AND TARGET SETTING

Students complete the Gap Analysis sheet:

MOCK EXAM GAP ANALYSIS

Name: _____

Total Marks: ____ / 50 (____ %)

Section A marks: ____ / 20

Section B marks: ____ / 18

Section C marks: ____ / 12

My strongest section: _____

My weakest section: _____

Questions I got completely wrong:

Topics I need to revise:

156. _____

157. _____

158. _____

My targets for Year 10:

159. _____

160. _____

161. _____

One thing I will do THIS WEEK to start improving:

37-40 min | PLENARY — CLASS OVERVIEW

Teacher shares class overview (anonymous):

- Class average: X%
- Strongest topic: (e.g., file handling)
- Weakest topic: (e.g., trace tables)
- "As we move into Year 10, we will build on these foundations."

Final words of encouragement. Collect gap analysis sheets.

EAL Support:

- Bilingual gap analysis sheet

- Sentence starters for targets: "I will improve my ____ by ____"

- Visual progress chart showing class distribution

- One-to-one check-in with EAL students during target setting
- Allow EAL students to write targets in Chinese if needed

Differentiation:

Emerging: Focus on celebrating progress. Review only Section A answers.

Set maximum 2 specific, achievable targets. Teacher discusses targets individually. Provide printed revision resources list.

Developing: Review all sections. Set 3 specific targets with actions.

Pair with secure student for peer explanation of difficult questions.

Secure: Review all sections. Set 3 ambitious targets. Research IGCSE

Computer Science syllabus independently. Prepare a revision plan for summer. Offer to tutor peers in weakest topics.

Resources Needed:

- Marked mock exam papers
- Projector with model answers
- Green pens for corrections
- Gap analysis sheets (printed)
- Class overview statistics (prepared by teacher)

MOCK EXAM MARK SCHEME

SECTION A — MULTIPLE CHOICE AND SHORT ANSWER (20 marks)

Question 1 (1 mark)

B) int()

Question 2 (1 mark)

B) 0 1 2

Question 3 (1 mark)

C) A program with a missing colon after an if statement

Question 4 (1 mark)

C) To handle errors without crashing the program

Question 5 (1 mark)

B) 'w'

Question 6 (2 marks)

Any TWO from:

- The data must be sorted / in order
- The data must be stored in a structure that supports random access (e.g., an array)
- The elements must be comparable

(1 mark each, any two valid)

Question 7 (2 marks)

Logic error: The program runs without crashing but produces incorrect or unexpected results. (1 mark)

Runtime error: The program starts running but crashes during execution due to an unexpected condition (e.g., division by zero, file not found). (1 mark)

Accept equivalent explanations.

Question 8 (3 marks)

with open("data.txt", "r") as file: (1 mark for with statement)

lines = file.readlines() (1 mark for readlines)

print(len(lines)) (1 mark for len and print)

Alternative using read and split: accepted if correct.

Alternative without with: deduct 1 mark.

Question 9 (2 marks)

data = "Alice,85,92,78".split(",") (1 mark for split)

name = data[0] (1 mark)

score1 = int(data[1]) (1 mark for int conversion)

(Maximum 2 marks — any two correct lines. Award 1 mark for split correct, 1 mark for correct indexing and int conversion)

Question 10 (3 marks)

Iteration	num	sum	num <= 4?	Output
Initial	1	0	-	-
1	1	1	True	-
After	2	1	True	-
2	2	5	True	-
After	3	5	True	-
3	3	14	True	-
After	4	14	True	-
4	4	30	True	-
After	5	30	False	-
End	5	30	-	30

(2 marks for correct iteration rows, 1 mark for final output = 30)

Partial credit for partially correct rows.

Question 11 (3 marks)

...

Alternative loop structures accepted. Deduct 1 mark if 'done' is written to file. Deduct 1 mark if newline not included.

Question 14 (6 marks)

(a) (4 marks)

with open("results.csv", "r") as file: (1 mark)

file.readline() # Skip header (1 mark)

total = 0

count = 0

for line in file: (1 mark)

data = line.strip().split(",")

score = int(data[1])

total = total + score

count = count + 1

average = total / count

print("Average: " + str(average)) (1 mark for average + print)

(b) (2 marks)

highest = -1 (0.5 marks)

best_student = ""

for line in file:

data = line.strip().split(",")

name = data[0]

score = int(data[1])

if score > highest: (0.5 marks)

highest = score

best_student = name (0.5 marks)

print("Best: " + best_student) (0.5 marks)

(If integrated with part (a) in a single loop, full marks awarded)

SECTION C — TESTING AND DEVELOPMENT (12 marks)

Question 15 (4 marks)

Test #	Description	Input Data	Expected Output	Test Type
1	Normal age	25	"Age accepted"	Normal
2	Minimum	0	"Age accepted"	Boundary

	boundary			
3	Maximum boundary	120	"Age accepted"	Boundary
4	Negative age	-5	"Invalid age"	Erroneous

(1 mark per correctly completed row. Accept any valid boundary values:

0, 120, or any value exactly on the boundary. Accept any valid erroneous: negative, > 120, non-numeric. Expected output must be appropriate.)

Question 16 (4 marks)

(a) (2 marks)

The function returns total (the sum) instead of the average. (1 mark)

For input [10, 20, 30], the function returns 60 instead of 20. (1 mark)

The line "return total" should divide total by the number of elements.

(b) (1 mark)

return total / len(numbers)

(Or equivalent: average = total / len(numbers); return average)

(c) (1 mark)

Any ONE valid strategy:

- Use print statements to check the return value
- Use a trace table to manually check the expected output
- Use test data where you know the expected answer (e.g., [10,20,30] → 20)
- Use Thonny debugger to step through and watch the return value
- Write a test plan with normal data and compare expected vs actual output

Question 17 (4 marks)

Any TWO stages explained with specific examples:

Analysis (1 mark + 1 mark for example):

Understanding the problem and identifying what the program needs to do, including inputs, outputs, and processing required. (1 mark)

Example: For attendance tracking, analysis would involve identifying that we need to record student names, dates, and present/absent status. (1 mark)

Design (1 mark + 1 mark for example):

Planning the solution using pseudocode, flowcharts, or structure diagrams to show how the program will work before coding. (1 mark)

Example: Drawing a flowchart showing the process of marking a student present or absent, including loops for multiple students. (1 mark)

Coding (1 mark + 1 mark for example):

Writing the actual program code in a programming language like Python. (1 mark)

Example: Writing Python functions to add a student, record attendance, and generate a report. (1 mark)

Testing (1 mark + 1 mark for example):

Checking the program works correctly using test data including normal, boundary, and erroneous cases. (1 mark)

Example: Testing with an empty class list, a class of exactly 1 student, and invalid inputs like negative student IDs. (1 mark)

Maintenance (1 mark + 1 mark for example):

Updating the program after release to fix bugs or add new features. (1 mark)

Example: Adding a new feature to generate monthly attendance reports after the initial program only tracked daily attendance. (1 mark)

Documentation (1 mark + 1 mark for example):

Writing comments in code (internal) and user/technical guides (external) to explain how the program works. (1 mark)

Example: Writing a user guide explaining how teachers can enter attendance data and generate reports. (1 mark)

(Total 4 marks — 2 marks per stage: 1 for description, 1 for specific example.

Maximum 2 stages assessed. If more than 2 given, mark best 2.)

TOTAL MARKS SUMMARY

Section A: 20 marks

Q1-Q5: 5 marks (1 each)

Q6: 2 marks

Q7: 2 marks

Q8: 3 marks

Q9: 2 marks

Q10: 3 marks

Q11: 3 marks

Section B: 18 marks

Q12: 6 marks

Q13: 6 marks

Q14: 6 marks

Section C: 12 marks

Q15: 4 marks

Q16: 4 marks

Q17: 4 marks

GRAND TOTAL: 50 marks

Grade Boundaries (suggested):

A*: 45-50 marks (90%+)

A: 40-44 marks (80-89%)

B: 35-39 marks (70-79%)

C: 30-34 marks (60-69%)

D: 25-29 marks (50-59%)

E: 20-24 marks (40-49%)

U: Below 20

END OF DOCUMENT

Victoria Park Academy Shenzhen

CIE IGCSE Computer Science (0478) — Year 9 Semester 2

Weeks 13-18: File Handling, Testing, Program Development & Mock Exam

Document prepared for:

- EAL learners (English as Additional Language)
- Mixed ability classes
- 40-minute lesson periods
- Thonny IDE (Python 3.x)

This document contains:

- 12 detailed lesson plans (2 per week for 6 weeks)
- 12 worksheets with questions (2 per week)
- Complete answer keys for all worksheets
- 1 full mock exam paper (50 marks, Paper 2 style)
- Complete mark scheme with grade boundaries
- Bilingual vocabulary support (English + Chinese + pinyin)
- Differentiation for every lesson (Emerging / Developing / Secure)
- EAL support strategies for every lesson